FinTechathon Shenzhen 2026 · International Track · Topic E (SME Finance Copilot)

CashBridge: A Governed, Evidence-Carrying Agent for SME Liquidity and Financing Decisions

Technical Documentation

---

20 October 2026

Abstract. Small and medium enterprises (SMEs) frequently fail not for lack of profit but for lack of timely liquidity, and the financing decision that would close the gap is spread across disconnected spreadsheets, document requests and lender comparisons. CashBridge is a working prototype that joins these steps into one workflow: it forecasts a 13-week cash position, verifies the receivables that would back a financing request, prices and stress-tests the available options, assembles a locked evidence pack, and passes the final external action to an authorised human. It also analyses and drafts commercial contracts and offers planning tools for project funding and green investment. Its central design commitment is that a language model never decides anything of financial consequence. Amounts, annualised costs, probabilities and policy outcomes are produced by deterministic or locally fitted models; the agent's authority is a finite-state machine bounded by a five-tier permission model; and every locked figure can be traced to a source record, a transformation and a hash. This document describes the architecture, the governed execution model, the financial and statistical methods, the evidence and security design, and the evaluation, and states plainly what the prototype does not claim.

Keywords: SME finance, receivables financing, liquidity forecasting, survival analysis, conformal prediction, conditional value-at-risk, agent governance, data provenance.

Scope notice. All companies, people, buyers, lenders and APIs in the submitted code are synthetic or mock. No real bank action occurs. Evaluation figures are team-produced and are not external certifications.

Contents

1 Introduction . . . 5
1.1 Problem statement . . . 5
1.2 Design principles . . . 5
1.3 Scope and non-claims . . . 5
2 Requirements . . . 5
3 System architecture . . . 6
3.1 Logical layers . . . 6
3.2 Module map . . . 7
3.3 Runtime modes and technology stack . . . 8
4 Governed agent execution . . . 8
4.1 Permission tiers . . . 8
4.2 The Finance Autopilot as a finite-state agent . . . 8
4.3 Action-bound one-time approval . . . 9
4.4 Separation of duties . . . 10
5 Financial methodology and algorithms . . . 10
5.1 Thirteen-week liquidity engine . . . 10
5.2 Receipt timing as a censored survival problem . . . 11
5.3 Calibrated forecast intervals . . . 12
5.4 Buyer payment-risk model . . . 12
5.5 Financing options: cost and annualised rate . . . 13
5.6 One funding timeline: drawdown, settlement and cash effect . . . 13
5.7 Monte Carlo scenario model . . . 14
5.8 Tail risk: value-at-risk and conditional value-at-risk . . . 15
5.9 Plan search and selection . . . 15
5.10 Feasibility certificate . . . 16
5.11 Learning from outcomes under human control . . . 16
5.12 Executive Inbox routing . . . 17
5.13 Financial-distress screening hint . . . 17
6 Document and data integrity . . . 17
6.1 Matching and reconciliation . . . 17
6.2 Data reliability gate . . . 18
6.3 Duplicate-financing control . . . 18
6.4 Click-to-source provenance . . . 18
6.5 Input defences . . . 19

7 Contract and document intelligence . . . 19
7.1 Design stance for untrusted documents . . . 19
7.2 Bounded extraction and local OCR . . . 19
7.3 Confidence-scored field extraction . . . 20
7.4 Rule-based contract risk analysis . . . 21
7.5 Contract–invoice reconciliation . . . 22
7.6 Deterministic agreement drafting and Word export . . . 22
7.7 Policy-driven contract-to-funding workflow . . . 23
7.8 Scope and verification of the module . . . 24
8 Planning and advisory tools . . . 24
8.1 Financial ratios and descriptive analysis . . . 25
8.2 Budget and marketing-spend scenario . . . 25
8.3 Project and bid funding feasibility . . . 25
8.4 Milestone cash waterfall . . . 26
8.5 Capital allocation planner . . . 26
8.6 Green-investment pre-screening and cash impact . . . 26
8.7 Conversational assistants under a number lock . . . 27
8.8 Collections: promises, reminders and the forecast . . . 28
9 Security architecture . . . 28
9.1 Identity, sessions and tenancy . . . 28
9.2 Tamper-evident audit log . . . 28
9.3 Encrypted evidence vault . . . 29
9.4 Lender integration boundary . . . 29
9.5 Privacy boundary and data lifecycle . . . 29
10 Interfaces and workflows . . . 30
10.1 REST interface . . . 30
10.2 End-to-end case . . . 31
10.3 Client . . . 31
10.4 Delivered screens . . . 31
11 Evaluation . . . 34
11.1 Metrics and statistical methods . . . 34
11.2 Functional, security and deployment evidence . . . 35
11.3 Forecasting and model validation . . . 35
11.4 Planning and decision-routing results . . . 36
11.5 What the evidence does not show . . . 36

12 Threat model and residual risk 36

13 Reproducibility 37

14 Limitations and future work 37

References 38

Appendix A. Agent tool allowlists 39

Appendix B. Principal configuration constants 40

Appendix C. Repository layout 40

1 Introduction

1.1 Problem statement

An SME can be profitable on paper and still miss payroll, because cash leaves on a contractual schedule while receipts arrive on a buyer's schedule. When a shortfall is foreseen, three questions follow in sequence: *when* does the balance turn negative; *which* receivables can credibly support a financing request; and *what* does each financing route really cost and how badly can it fail. In practice each question is answered in a different tool, by a different person, with different evidence, and the lender receives a pack whose figures cannot be traced back to their source.

The prototype also contains a contract- and document-intelligence module (OCR, clause and amount analysis, invoice reconciliation, deterministic drafting) and a set of planning tools for capital allocation, project and bid funding, and green investment. These share the same evidence and approval controls and are described in Sections 7 and 8.

CashBridge treats the three questions, and the approval that follows them, as one accountable workflow. The contribution of the prototype is not a single model but the way models, controls and evidence are composed so that an autonomous agent can prepare a financing decision without ever being able to commit the firm.

1.2 Design principles

Principle	Engineering consequence
Financial facts before language	Money, annualised cost, probability bands and policy results are produced by local code. Text is a rendering of results, never a source of them.
Agency with bounded authority	The Autopilot selects only from an authorised set of next steps. A guard that is independent of the planner can reject any proposal.
Evidence travels with the decision	A locked figure carries its formula, its input records, the source file hash and any later correction, so a reviewer can re-derive it.
Humans own irreversible decisions	External release needs a named Director and an action-bound one-time code. No agent holds a money-movement tool.

Table 1. The four principles and how each is enforced in the implementation.

1.3 Scope and non-claims

The system is a sandboxed prototype operating on a synthetic SME case (a timber exporter with two preparers, two approvers and a mock lender). The following are **not** claimed: live bank feeds, production credit eligibility, binding electronic signatures, actual disbursement, enterprise single sign-on, a certified KYC/AML process, or an independent penetration test. Forecast and risk models have been validated on public data and synthetic cases; they have not been validated on the cash flows of real firms.

2 Requirements

Six engineering requirements drive the design. Each is paired with an acceptance signal that can be observed in the running system or in the archived test output.

ID	Requirement	Acceptance signal
R1	Detect a cash gap in time	13-week weekly cash balance, first shortfall week, driver attribution and alerts.
R2	Ground every financing option	Receivable three-way matching, an eligibility gate, and cost, tenor and annualised cost for each option.

ID	Requirement	Acceptance signal
R3	Show downside, not only price	Scenario paths, negative-week probability, $CVaR_{95}$, a feasible plan and the binding policy limit.
R4	Complete an accountable case	Accountant prepares; Director releases with OTP; lender reviews evidence; mock decision returns to the SME.
R5	Keep autonomous authority bounded	Tier checks L0 to L4, per-agent allowlists, a refusal log, and no money-movement tool.
R6	Make explanations verifiable	Hash-chained audit log, evidence statuses, click-to-source provenance, versioned encrypted documents.

Table 2. Requirements and acceptance signals.

3 System architecture

3.1 Logical layers

The implementation separates a control plane from the computation it governs. A request enters a FastAPI gateway that verifies a signed session, the caller's role and tenant. It is then routed to a typed workflow in the coordinator, not to a free-form planner. The coordinator calls local specialist engines and writes every consequential step to a tamper-evident audit log. The logical diagram does not imply one deployed service per agent: all components run in one Python process behind the API.

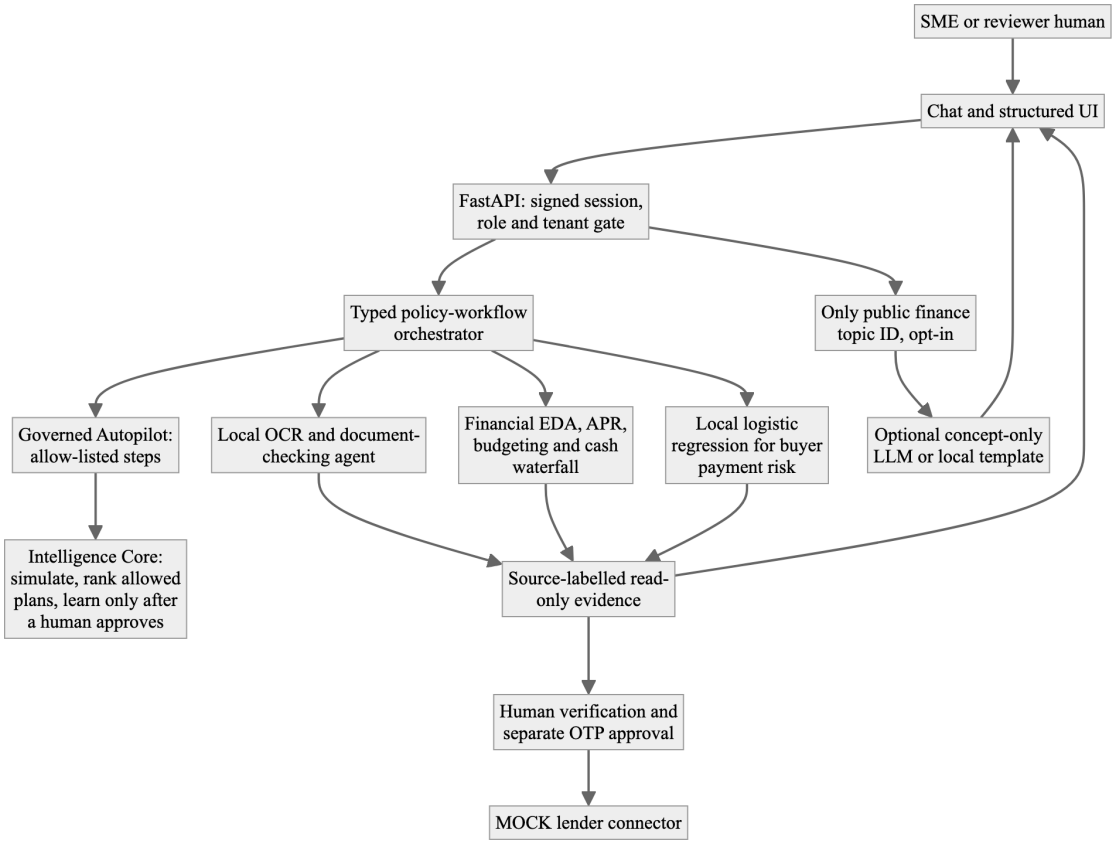


Figure 1. Logical architecture, reproduced from the project README (Mermaid source rendered unchanged; line breaks were inserted inside node labels for legibility). Control passes from the human through the signed-session gate to the typed workflow orchestrator and the local agents. The optional language model receives only a public topic identifier and has no path to case data or tools.

3.2 Module map

Layer	Modules	Responsibility
Gateway	api/main.py, api/lender_adapters.py	About 90 REST routes; session and role checks; quote adapters with signed webhooks.
Orchestration	agents/coordinator.py, autopilot.py, intelligence_core.py	Case state, workflow transitions, bounded planning, pack lifecycle.
Contract intelligence	contracts/intelligence, extraction, docx_export; agents/decision_copilot	Bounded OCR, clause and amount analysis, contract–invoice reconciliation, deterministic drafting, policy workflows.
Planning tools	core/financial_intelligence, project_cashflow, capital_planning, green_finance	Ratios, budget and bid stress, milestone waterfall, capital allocation, green pre-screening and cash impact.
Specialist agents	document_agent, buyer_risk_agent, collections_agent, cfo_assistant, concept_explainer	Local extraction, scoring, reminder drafting, constrained explanations.
Finance core	core/forecasting_engine, payment_timing, conformal, funding_evaluator, funding_schedule, funding_coverage, scenario_simulator	Forecast, receipt timing, calibrated intervals, cost, settlement and coverage.
Integrity	core/matching_engine, financing_gate, data_reliability, source_attestation, provenance, invoice_registry, period_lock	Matching, fail-closed gate, signed attestations, lineage, duplicate detection.
Security	security/auth, permission_gatekeeper, otp_verifier, audit_log, injection_guard, number_check, bec_fraud_detector, pii_masker	Identity, authorisation, approvals, tamper evidence and input defences.
Workspace	workspace/store, evidence_vault, corrections	Versioned requests, messages, encrypted evidence, reviewed corrections.
Client	frontend/src (React 18, TypeScript, Vite)	Feature modules per workspace screen; no business logic is trusted in the browser.

Table 3. Source modules by architectural layer.

The deterministic analysis engines added in the financial and contract intelligence extension (financial ratios, contract rules with local OCR, project and bid stress, green-upgrade payback) hang off the same gateway, as shown in Figure 2.

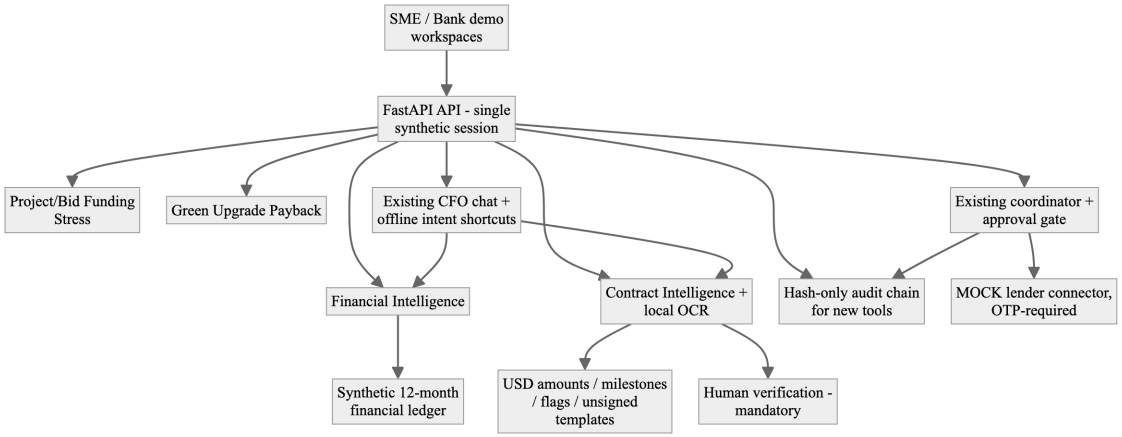


Figure 2. Deterministic analysis engines and their audit path, reproduced from docs/INTELLIGENCE_EXTENSION.md (Mermaid source rendered unchanged apart from label line breaks). No engine calls a language model; human verification of contract output is mandatory.

3.3 Runtime modes and technology stack

The backend is Python with FastAPI, Pydantic v2 for typed inputs, NumPy for numerical work, and scikit-learn when installed (an equivalent NumPy Newton solver is used otherwise, and a test asserts that both agree). Cryptography uses the *cryptography* package (AES-256-GCM) and the standard library (HMAC-SHA256, SHA-256). Persistence is SQLite in WAL mode. The client is React 18 with TypeScript, built with Vite. Reports are generated with fpdf2 and python-docx.

Two deployment modes exist. In **DEMO** mode, one-click demo accounts and a public demo OTP secret keep the system runnable with zero configuration. In **PRODUCTION** mode the process refuses to start if any demo shortcut would remain active (a default OTP secret, a session secret shorter than 32 characters, no configured accounts, or no vault key); the data gate is forced on; and demo login, reset and tamper demonstrations are disabled. An unrecognised mode string (for example a misspelling) is treated as PRODUCTION, so that a configuration error fails closed.

4 Governed agent execution

4.1 Permission tiers

Every tool is registered with exactly one tier in `config/tool_allowlist.py`. A permission gatekeeper checks the tier, the calling agent's allowlist and, for writes, whether a human is required, and it logs every refusal.

Tier	Class	Representative tools	Boundary
L0	Read	`read_bank_statements, read_documents, read_payment_history`	Session, tenant and role checked.
L1	Analyse	`run_forecast, three_way_match, rank_funding_options, score_buyer_risk, plan_liquidity`	Must be on the agent's allowlist; no outside effect.
L2	Internal write	`ledger_write, lock_period, propose_model_update, record_plan_decision`	Deterministic, or a named human; model activation is always human.
L3	External	`submit_pack, send_email`	Coordinator only; Director OTP bound to the exact payload.
L4	Money movement	`transfer_funds, update_payee, make_payment`	Recognised only so it can be refused; on no allowlist.

Table 4. The five-tier permission model.

Allowlists are per agent. The coordinator alone holds the two L3 tools. The buyer-risk agent can read payment history and score risk and nothing else; the Intelligence Core may *plan*, *simulate* and *propose* a model update but cannot activate one. The lender's reviewer can read documents and record a credit decision but cannot touch the pack.

4.2 The Finance Autopilot as a finite-state agent

The Autopilot observes case state, selects the next step, runs a tool, records the result as evidence, and stops or escalates when the evidence does not support progress. Its transitions are fixed by the table `ALLOWED_NEXT`.

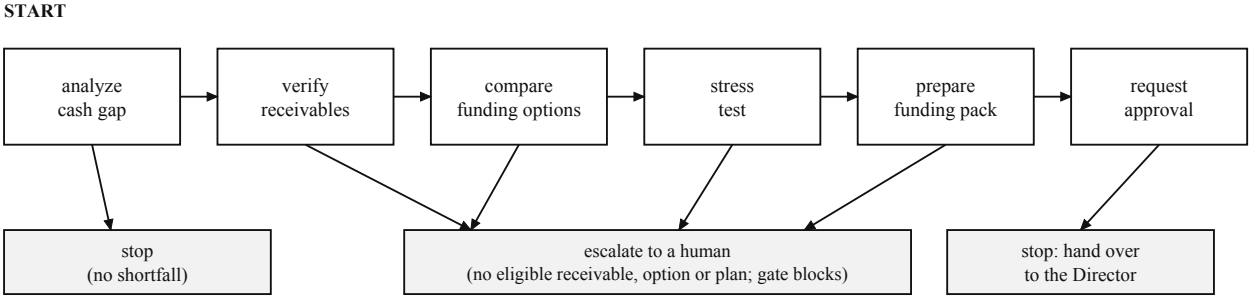


Figure 3. Authorised transitions of the Finance Autopilot. Terminal states stop or escalate. The agent has no transition to release a pack.

The default planner is a deterministic policy (`deterministic-policy-v1`). A different planner, including a language-model planner, can be injected through a callback, but it is held to the same guard: if its proposal differs from the policy choice, the proposal is classified and refused with a reason (money movement is never available to an agent; external actions require the Director; the step is not an allowed successor; or the evidence does not yet support the step). The refusal is appended to the audit log and the safe policy step is used. A planner that raises an exception cannot halt the run.

Each step registers claims with an evidence status drawn from six values: VERIFIED, CALCULATED, MODEL_ESTIMATE, ASSUMPTION, NEEDS_HUMAN and NOT_VERIFIABLE_HERE. A reader can therefore distinguish a confirmed input from an engine-computed amount, a model estimate, an assumption, and a gap that the system cannot check.

4.3 Action-bound one-time approval

An L3 action requires a time-based one-time code (RFC 6238) from the Director. The code is not a generic login factor; it is bound to the action by hashing the action type with the canonical JSON of its payload:

$$action_hash = SHA\text{-}256(\ action_type \parallel canonical_json(payload)\ ) \tag{1}$$

An approval is accepted only if the code is current, matches the challenge's *action_hash*, has not been consumed, and arrives within 120 seconds. Approving pack A therefore cannot authorise pack B, and altering the payload after approval invalidates it. A challenge is voided after five wrong attempts. The four error classes returned are *invalid_code*, *expired*, *replayed* and *payload_mismatch*. The coordinator additionally refuses an approval from the user who prepared the pack, enforcing segregation of duties in two places (permission matrix and coordinator).

Code generation and guess probability. The one-time code is the time-based construction of [1, 2]. With shared secret K, step $X = 30$ s and time t, the counter is $c = \mathrm{floor}(t / X)$ and

$$code(\ t\ ) = (\ Truncate(\ HMAC_K(\ c\ )\ )\ mod\ 2^{31}\ )\ mod\ 10^6 \tag{2}$$

where Truncate extracts four bytes at an offset taken from the low nibble of the digest, giving a six-digit code. The verifier accepts the current step and one step either side (clock drift), so three of the 10^6 values are valid at any time. A random guess therefore succeeds with probability at most 3×10^{-6}, and because a challenge is voided after five failures, an attacker without the secret succeeds on a given challenge with probability at most 1.5×10^{-5}. Replay protection is separate: the pair (action hash, code) is remembered once consumed. The approval binds to the action through the hash above and not through the secret alone, so a valid code for one payload is useless for another.

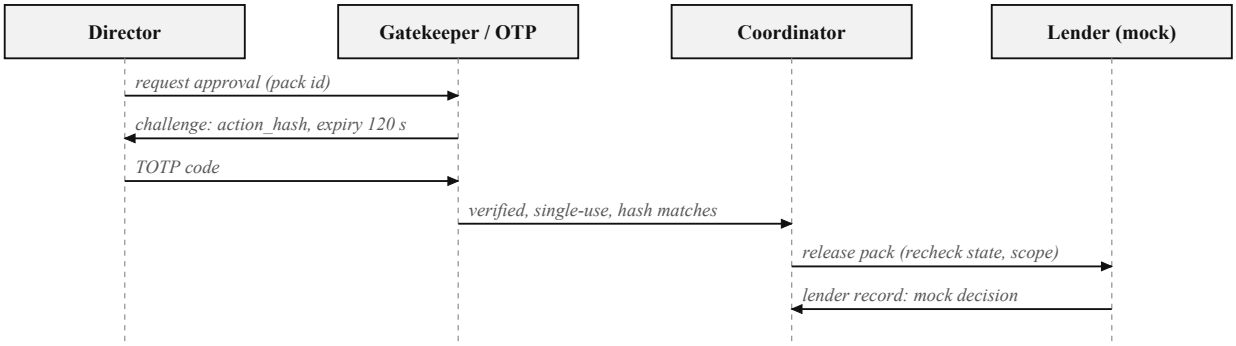


Figure 4. Release sequence. The one-time code is validated against the exact payload hash before the coordinator rechecks case state and lender scope.

4.4 Separation of duties

Actor	May	May not
Accountant (sme_accountant)	Analyse; supply evidence; prepare or rework a pack; propose a correction.	Approve own release; act as lender.
Director (sme_director)	Review or reject; confirm release by OTP; approve governed model changes and corrections.	Prepare; bypass the data gate, OTP or money prohibition.
Lender reviewer (bank_reviewer)	Read released cases; request and open evidence; record a mock decision.	Access unreleased cases; disburse funds.
Finance Autopilot	Analyse, verify, price, stress-test, draft, request approval.	Submit a pack; approve; lend; transfer.

Table 5. Role capabilities, taken from the permission matrix in security/auth.py.

5 Financial methodology and algorithms

This section specifies each algorithm as implemented, with its notation, assumptions, parameter values and the reason for the design choice. Parameter values are quoted from the source. Where the implementation deliberately departs from a textbook method, or where a guarantee holds only approximately, the text says so. Throughout, t indexes forecast weeks 1, …, T with $T = 13$, n indexes Monte Carlo paths 1, …, N, and i indexes receivables. I(·) denotes the indicator function, equal to 1 when its argument is true and 0 otherwise; the interest amount is written I with a subscript.

5.1 Thirteen-week liquidity engine

Let C_0 be the opening cash. The deterministic forecast is the recurrence

$$C_t = C_{t-1} + R_t - O_t + F_t, \quad t = 1, \ldots, T \tag{3}$$

where R_t is expected receipts, O_t is scheduled outflows (weekly fixed costs, supplier deposits, and payroll in any week that contains a month start), and F_t is the net effect of a chosen financing plan. Receipts are the sum over open invoices of the amount times the probability that payment falls in week t (Section 5.2). Confirmed but unbilled orders enter with their stated ship confidence and are never eligible to be pledged. A week with C_t below the buffer b = USD 10,000 is flagged *at risk*, and a week with $C_t < 0$ a *shortfall*; the first such week is reported with a driver attribution that ranks the receipts and payments responsible for the gap and folds minor late drivers into a single line.

Working-capital indicators use standard definitions: days sales outstanding DSO = (average receivables / revenue) × D, days inventory outstanding DIO = (average inventory / COGS) × D, days payables outstanding DPO = (average payables / COGS) × D, and the cash conversion cycle CCC = DIO + DSO −

DPO, with D the number of days in the ledger period.

5.2 Receipt timing as a censored survival problem

Motivation. The earlier rule put the on-time share p of an invoice on its due date and the remainder a fixed four weeks later. Real delay is neither binary nor fixed. More importantly, an invoice that is still unpaid at the forecast origin is not "late by an unknown amount"; it is *right-censored*: all that is known is that the payment delay exceeds the age of the invoice. Discarding such invoices, or counting them as late, biases the estimated delay distribution (downward and upward respectively). Survival analysis handles censoring correctly [3, 4].

Discrete delay variable. Let L be the payment date minus the due date, in days, so that $L < 0$ is early payment. Delay is discretised into cells $c = 0, 1, \ldots, K$, where cell 0 collects payments 60 or more days early, cells 1 to 180 cover days −59 to +120, and cell K is an absorbing "later than 120 days" tail. At forecast origin o only point-in-time information is used: an invoice contributes an *event* in cell c_i if it was paid on or before o; otherwise it contributes a *censoring* in the cell of its current age $o - \text{due}_i$ (provided that age is at least −60 days). Payments after o are never visible.

Kaplan–Meier hazard. With d_c events in cell c and m_c censorings in cell c, the number at risk at the start of cell c is the number of invoices whose event or censoring lies in a cell at or after c:

$$n_c = \sum_{c' \geq c} (d_{c'} + m_{c'}), \quad h_c = d_c / n_c, \quad h_K = 1 \tag{4}$$

The survival function and the probability mass of paying exactly in cell c follow:

$$S_c = \prod_{c' < c} (1 - h_{c'}), \quad f_c = S_c \cdot h_c, \quad \sum_c f_c = 1 \tag{5}$$

This is the product-limit estimator of Kaplan and Meier [3] in discrete time, which is the non-parametric maximum-likelihood estimator of the survival function under independent right-censoring. Setting $h_K = 1$ closes the support so that every invoice is eventually paid with probability one; mass beyond the horizon is reported separately as "expected after the horizon" and never counted as cash inside the 13 weeks.

Conditioning on age. For an invoice already unpaid through relative day a at the origin, the predictive mass is renormalised on the event $L > a$:

$$f_c(a) = f_c \cdot I(c > \text{cell}(a)) / \sum_{c' > \text{cell}(a)} f_{c'} \tag{6}$$

If no mass remains (the invoice is older than anything observed), all mass is placed in the tail cell, which conservatively removes it from the horizon.

Expected receipts. For invoice i of amount V_i due on day u_i, a payment in cell c arrives on day $u_i + l(c)$ and falls in week w = floor(max(0, day − start) / 7) (zero-based). Expected weekly receipts are

$$R_t = \sum_i V_i \sum_c f_c(a_i) \cdot I(w_i(c) = t) \tag{7}$$

and a Monte Carlo variant draws, independently per invoice, a cell from the categorical distribution $f(a_i)$, which is used for the forecast's own simulation band.

Pooled versus buyer-specific estimation. The shipped estimator is the *pooled* Kaplan–Meier hazard over all buyers. A buyer-specific variant was implemented and evaluated but is not used. It shrinks each buyer's hazard toward the pooled hazard $h^{(0)}$ with a Beta prior of strength $k = 8$ on each daily hazard,

$$h_c^{(b)} = (d_c^{(b)} + k \cdot h_c^{(0)}) / (n_c^{(b)} + k) \tag{8}$$

which is the posterior mean of a Beta($k\ h^{(0)}$, $k(1 - h^{(0)})$) prior updated with d events in n trials (empirical-Bayes shrinkage in the sense of [5]). On the public sample the pooled estimator was better (WAPE 20.9% against 22.2%), which is why the simpler model is used; with about 25 invoices per buyer the extra variance of buyer-specific hazards is not repaid. The on-time probability fed to the planner is the mass up to a grace period of three days, $p = \sum_{c \leq \text{cell}(3)} f_c$.

Accuracy measures. Forecast quality is the weighted absolute percentage error and bias, aggregated over origins j and weeks t [6]:

$$WAPE = \sum_{j,t} | \rho_{j,t} - R_{j,t} | / \sum_{j,t} R_{j,t}, \quad Bias = \sum_{j,t} (\rho_{j,t} - R_{j,t}) / \sum_{j,t} R_{j,t} \tag{9}$$

where R is realised receipts from invoices open at the origin. WAPE is preferred to MAPE because weeks with near-zero receipts would otherwise dominate; it is undefined, and reported as such, at a horizon that has no receipts.

5.3 Calibrated forecast intervals

A model's own simulation band is only as honest as the model: on the public sample the Monte Carlo P10–P90 band covered 87.1% against a nominal 80%. Split conformal prediction [7, 8, 9] replaces it with an interval whose coverage is checked against realised errors. For each horizon h, collect past absolute errors (nonconformity scores) $s_1, \ldots, s_n$ that were realised by the origin, and take the finite-sample empirical quantile

$$q_h = s_{(k)}, \quad k = ceil((n + 1)(1 - \alpha)), \quad interval = [\rho_h - q_h, \rho_h + q_h] \tag{10}$$

with $q_h = +\infty$ when $k > n$ (too few scores for the level). If the scores are exchangeable with the future error, $1 - \alpha \le \mathrm{P}(R \text{ in interval}) \le 1 - \alpha + 1/(n + 1)$ [7, 8]. Weekly forecast errors across overlapping origins are serially dependent, so exchangeability does not strictly hold and the guarantee is approximate; this is why coverage is *measured* out of sample (80.6% external, 80.1% synthetic) instead of assumed.

Adaptive conformal inference. To cope with drift, ACI [10] tracks a working miscoverage level updated online,

$$\alpha_{t+1} = \alpha_t + \gamma (\alpha - err_t), \quad err_t = I(R_t \text{ outside its interval}) \tag{11}$$

which guarantees, for any data sequence, that the average miss rate over T steps differs from α by at most $(\max\{\alpha_1, 1 - \alpha_1\} + \gamma) / (\gamma T)$ [10]. With $\alpha = 0.2$, $\gamma = 0.02$ and a 200-score window the bound is loose for short histories. It reached 81.0% on 2,466 public invoices but only 74.0% on the 185-invoice synthetic history, so it is reported and *not* used; the product uses plain split conformal.

5.4 Buyer payment-risk model

Specification. For invoice i from buyer b, the feature vector is $\mathbf{x}_i$ = (one-hot(b), terms_i / 30, V_i / $\text{median}_b(V)$), and the model is logistic regression for the event "paid within three days of due" ($y = 1$):

$$P(y_i = 1 \mid \boldsymbol{x}_i) = \sigma(\beta_0 + \boldsymbol{\beta}' \boldsymbol{x}_i), \quad \sigma(z) = 1 / (1 + e^{-z}) \tag{12}$$

Estimation. Coefficients minimise the L2-penalised negative log-likelihood with the intercept unpenalised and $C = 10$:

$$J(\boldsymbol{\beta}) = - \sum_i [y_i \ln p_i + (1 - y_i) \ln(1 - p_i)] + (1 / 2C) \| \boldsymbol{\beta}_{1:} \|^2 \tag{13}$$

J is strictly convex, so Newton–Raphson (equivalently iteratively reweighted least squares [11]) converges to the unique minimum: $\boldsymbol{\beta} := \boldsymbol{\beta} - \mathbf{H}^{-1}\mathbf{g}$, with $\mathbf{g} = \mathbf{X}'(\mathbf{p} - \mathbf{y}) + \mathrm{diag}(\boldsymbol{\lambda})\boldsymbol{\beta}$ and $\mathbf{H} = \mathbf{X}'\mathbf{W}\mathbf{X} + \mathrm{diag}(\boldsymbol{\lambda})$, where $\mathbf{W} = \mathrm{diag}(p_i(1 - p_i))$, $\lambda_j = 1/C$ for penalised coordinates and 0 for the intercept. The scikit-learn solver is used when installed; the NumPy implementation of the same objective is used otherwise, and a test asserts that the two agree.

Safeguards. Fitted probabilities are clipped to [0.02, 0.98], so the model cannot state certainty from one firm's history; a buyer absent from the history receives the portfolio estimate and is labelled; a history with a single outcome falls back to a smoothed constant; and inputs outside the training range raise an *out_of_domain* warning. Concentration risk is flagged when one buyer holds more than 40% of open receivables.

Validation. Models are evaluated with walk-forward splits, so that each prediction uses only earlier invoices. Discrimination is measured by the area under the ROC curve, AUC = $\mathrm{P}(s^+ > s^-) + \tfrac{1}{2}\mathrm{P}(s^+ = s^-)$, the Mann–Whitney statistic [12]; overall accuracy by the Brier score [13] = $(1/n)\sum (p_i - y_i)^2$; and calibration by the expected calibration error [14], ECE = $\sum_m (|B_m| / n) \,|\, \mathrm{mean}(y; B_m) - \mathrm{mean}(p; B_m) \,|$, over probability bins B_m. On the synthetic history the Brier score is 0.0906 with buyer identity and 0.1481 without it.

5.5 Financing options: cost and annualised rate

All monetary arithmetic uses `Decimal` with half-up rounding to the cent. Simple interest on principal P at nominal rate r per annum for d days uses an Actual/365 basis, $I = P\,r\,d\,/\,365$. The reported annualised cost is

$$APR = (K / N) \times (365 / d) \tag{14}$$

where K is total financing cost, N is net funds actually received and d is days outstanding. The four products specialise this as follows.

Product	Net funds N	Total cost K	Annualised cost
Invoice factoring (invoice V, advance rate a, fee rate f, rate r)	$A = aV$	$fV + A\,r\,d\,/\,365$	$(K / A)(365 / d)$
Bridge loan (principal P, upfront fee rate φ)	$P(1 - \varphi)$	$\varphi P + P\,r\,d\,/\,365$	$(K / N)(365 / d)$
Early-payment discount (rate δ, terms δ/T net M)	$V(1 - \delta)$	δV	$\delta/(1 - \delta) \times 365 / (M - T)$
Overdraft (drawn amount P)	P	$P\,r\,d\,/\,365$	r (by construction)

Table 6. Cost formulae implemented in core/funding_evaluator.py.

Worked example (illustrative values, not catalogue terms). Factor an invoice V = 100,000 for d = 60 days with a = 80%, f = 1% and r = 12%. Then A = 80,000; fee = 1,000; interest = 80,000 × 0.12 × 60 / 365 = 1,578.08; K = 2,578.08; and APR = (2,578.08 / 80,000)(365 / 60) = 19.60%. The early-discount formula follows from requiring that giving up δV to be paid $M - T$ days sooner earns the same simple annual rate; for 2/10 net 30 it gives (0.02 / 0.98)(365 / 20) = 37.24%, the figure shown for the demonstration case in Figure 10, against 17.12% for factoring and 18.24% for the bridge loan. Because these are simple-interest figures, an effective compounded annual rate $(1 + K/N)^{365/d} - 1$ is higher (21.28% in the worked example); the product reports the simple figure, labelled as CashBridge's internal convention and not a statutory APR, because that is the convention used by the quotes it compares.

Eligibility. A factoring option is ineligible, with the reason recorded, when the invoice failed verification, its tenor exceeds the lender's maximum (120 days in the mock catalogue), or it is below the minimum invoice size. Options are ranked by ascending APR and each carries a *why not cheaper* statement quantifying the gap in annual percentage points.

5.6 One funding timeline: drawdown, settlement and cash effect

A *FundingSchedule* fixes the quote date, drawdown date and expected settlement date once; the quote, the coverage chart and the settlement scenarios all read it. For factoring the cash effect is derived as follows [15]. The buyer pays the factor, which retains the advance, the fee and interest accrued to the payment date, and remits the remainder (the rebate) to the SME. Suppose the base forecast already contains the receipt, and let s_k be the probability share of the invoice expected to be paid on date t_k ($\sum s_k = 1$). Relative to the base forecast the SME then experiences

$$\Delta C(\text{drawdown}) = +A, \quad \Delta C(t_k) = -s_k V + s_k (V - A - F - I_k) = -s_k (A + F + I_k) \tag{15}$$

where the first term in the second expression removes the receipt that the factor now collects and the second adds the rebate share. Summing over all k and over a horizon that contains every receipt gives a total of $A -$

$(A + F + I_E) = -(F + I_E)$, the expected financing cost, to the cent: nothing is counted twice. A later drawdown re-prices interest for the days actually financed, and with drawdown on the quote date the schedule reproduces the quote exactly (a tested invariant).

Recourse. With recourse the SME repays the factor if the buyer never pays, which is modelled as default established 90 days after the due date; with non-recourse the factor bears that loss and the schedule records the difference.

5.7 Monte Carlo scenario model

Why simulate. The forecast gives the expected balance, but a lender and a Director need the probability that a plan fails and how badly. The simulator therefore draws $N = 2{,}000$ (default) paths from a documented, versioned world model and replays every candidate plan on the *same* paths. Using common random numbers makes the *difference* between two plans far less noisy than either estimate alone, because the sampling error shared by both cancels [16]. Draws are seeded, so identical inputs, parameters and seed reproduce identical paths.

Generative model. Each receivable i belongs to buyer $b(i)$ and has on-time probability p_i from Section 5.2 or 5.4. For each path:

- a buyer *slowdown* indicator $Z_b \sim \text{Bernoulli}(\pi)$, $\pi = 0.05$, drawn once per buyer per path so that a buyer's invoices slow down *together* (this correlated risk is what a single expected path conceals);
- the effective on-time probability $q_i = \sigma(\text{logit}\, p_i + \delta_b) \times (m \text{ if } Z_b = 1 \text{ else } 1)$, with learnable per-buyer shift δ_b and slowdown multiplier $m = 0.5$;
- payment week W_i = due week if on time (probability q_i), otherwise due week + $\lambda + e\, Z_b$ with late delay $\lambda = 4$ weeks and slowdown extra delay $e = 2$ weeks;
- for a confirmed order, shipment on plan with probability $\min(1, c_i\, s)$, where c_i is the stated ship confidence and s a learnable scale, otherwise an additional 3-week slip; an unshipped order is never cash in the week planned;
- one cost factor per path $\kappa \sim \text{LogNormal}(-\sigma_c^2/2, \sigma_c^2)$ with $\sigma_c = 0.04$, scaling all outflows (the offset gives $E[\kappa] = 1$), and one FX factor $\varphi \sim N(1, 0.02^2)$ scaling every foreign-currency receipt;
- a supplier accepts a requested deposit reduction with probability 0.6.

Path n then has weekly net cash $x_{n,t}$ = inflows − outflows + plan flows, closing balance $C_{n,t} = C_0 + \sum_{u \le t} x_{n,u}$, trough $T_n = \min_t C_{n,t}$ and shortfall $S_n = \max(0, -T_n)$. The goal probability for a floor (zero, or the buffer) is the Monte Carlo estimate

$$P_{goal}(A) = (1/N) \sum_n I(T_n(A) \ge floor), \quad SE = \sqrt{(P(1-P)/N)} \tag{16}$$

so that with $N = 2{,}000$ the standard error never exceeds 1.1 percentage points. A plan is a list of actions from a closed set; the action flows are, for factoring with advance A drawn in week w_d against invoice i,

$$\Delta C_n(w_d) = +A, \quad \Delta C_n(w_{s,n}) = -(A + F + I_n), \quad w_{s,n} = max(W_{i,n}, w_d) \tag{17}$$

$$I_n = A \cdot r \cdot max(7, 7(w_{s,n} - w_d) + 2)/365 \tag{18}$$

so a path on which the buyer pays late makes the financing dearer on that path, which is how the model captures the way delay inflates cost. A bridge loan adds $N = P -$ fee at drawdown and subtracts $P + I$ after ceil(d/7) weeks when repayment falls inside the horizon, and a deposit cut releases the saving only on paths where the supplier agrees. The module computes balances only and has no side effects.

World-model versioning. Parameters are held in an immutable record whose version identifier is the first ten hexadecimal characters of the SHA-256 of its sorted JSON; every result states the version that produced it, so a changed assumption is visible as a changed identifier.

5.8 Tail risk: value-at-risk and conditional value-at-risk

For a loss variable S with distribution function F_S and level $\alpha = 0.95$, value-at-risk is $\mathrm{VaR}_\alpha = \inf\{ z : F_S(z) \geq \alpha \}$ and conditional value-at-risk is the mean loss beyond it. Rockafellar and Uryasev [17] show that

$$CVaR_\alpha(S) = \min_\zeta \{ \zeta + (1/(1-\alpha))\, E[(S-\zeta)^+] \}, \quad \text{minimiser } \zeta^* = VaR_\alpha \tag{19}$$

which on a sample of N paths reduces to the average of the ceil$((1-\alpha)N)$ largest shortfalls; this is exactly what the implementation computes (the worst 100 of 2,000 paths). CVaR is a coherent risk measure [18], so unlike VaR it is subadditive and it is always at least the 95th-percentile shortfall; it also penalises the *size* of the bad outcomes and not only their frequency. A shortfall variable is used, not the closing balance, so that the measure has the units of dollars of cash missing in the worst 5% of futures.

5.9 Plan search and selection

Candidate generation. Candidate plans are enumerated, not generated by a model. For each draw week in {first week with P(negative) ≥ 10%, base-case first negative week}, the usable invoices are those with at least 14 days to due, at least the lender minimum size, and $p_i \geq 0.5$, limited to the largest few. The plan set contains the empty plan; every combination of one, two or three factored invoices; the bridge loan alone; the loan with one or two factored invoices; and each of these with the supplier-deposit reduction. The count is bounded by $\sum_{k=1}^{3} C(m, k)$ plus loan combinations, which for $m \leq 12$ usable invoices is in the hundreds; the demonstration case scores 250 plans. Exhaustive enumeration over a small closed set is chosen over an integer-programming formulation because it is auditable: the ranked list, including the plans that lost and why, can be shown to the Director.

Independent validation. Every candidate, including any plan injected by a caller, passes *validate_plan*, a checker separate from the planner. It requires (i) every action to be in the closed set; (ii) no more than three packs; (iii) every pledged invoice to be verified, unpledged and at least 14 days from due; (iv) $p_i \geq 0.5$ to limit adverse selection; and (v) the over-borrowing bound

$$\sum_{a \text{ in } A} funding(a) \leq 1.5 \cdot S^0_{95} \tag{20}$$

where S^0_{95} is the 95th percentile of the no-action shortfall. Plans that fail are returned with their violations.

Optimisation problem. Selection is a sample-average approximation [19, 20] of a chance-constrained programme [21]:

$$\min_A \; J(A) = E[K(A)] + \lambda \cdot CVaR_{95}(S(A)) \quad \text{subject to} \quad P(T(A) \geq floor) \geq \tau, \quad A \text{ valid} \tag{21}$$

with $K(A)$ the realised financing cost on a path, $\lambda = 0.02$ and τ chosen from {0.80, 0.85, 0.90, 0.95} (default 0.90 for a zero floor and 0.80 for the buffer floor). The weight λ is a fixed policy value derived, not tuned: a 5% tail probability times an assumed 40% cost of emergency funding in that tail gives 0.02, that is, the planner will pay up to USD 2 of expected cost to remove USD 100 of tail shortfall. Ties are broken lexicographically by cost, fewest packs, no need for supplier agreement, then plan identifier, so the choice is deterministic. The default objective is *risk-adjusted* (J above); a *cheapest* objective minimises $E[K]$ alone and was the baseline in the benchmark.

When the goal is unreachable. If no valid plan meets τ, the planner does not weaken it. It identifies the best achievable probability p^* and chooses the cheapest plan whose probability lies within two standard errors of it, $2\sqrt{(p^*(1-p^*)/N)}$, because differences smaller than the simulation noise should not be paid for. The status is reported as *GOAL_NOT_REACHABLE*.

Stability of the recommendation. To avoid asking the SME to switch plans over simulation noise, an incumbent plan is retained when it meets the same goal status, its probability is within that noise tolerance of the new best, and its expected cost is at most 1.10 times the new best plus USD 50.

Robust variant. For the *robust* objective the goal probability is replaced by its worst case over a finite ambiguity set of adverse worlds $W_1, \ldots, W_J$:

$$p_{robust}(A) = \min_{j=1..J} P_{W_j}(T(A) \geq floor) \tag{22}$$

The set comprises: any single buyer's odds of paying on time halving (a logit shift of ln 0.5); all buyers' odds falling by a third together (ln 2/3); and confirmed orders shipping at 85% of stated confidence. These are fixed policy values. The construction is a finite-scenario instance of distributionally robust optimisation [22, 23] and of robust optimisation more generally [24], with the same seed used in every world so that differences reflect the world and not the draw.

5.10 Feasibility certificate

When the goal is not reached, the Director needs to know whether the obstacle is financing availability or policy. The certificate begins with an exact result. Adding an amount X of free cash in week 1 lifts every weekly balance on every path by X, hence the trough by X. So $P(T + X \geq \text{floor}) \geq \tau$ holds exactly when X is at least the τ-quantile of the required top-up:

$$X^* = Q_\tau(\max(0, floor - T_n)) \tag{23}$$

No financing that arrives later, costs money, or is repaid inside the horizon can reach the goal with a smaller amount, so X^* is a lower bound on the cash any plan must supply. It is compared with the over-borrowing cap $1.5\, S^0_{95}$ and with the total funding that all eligible sources could provide. A set of probes then relaxes one limit at a time on the *same* scenarios (more packs inside the cap, free cash at the cap, every eligible source at once, the recommended plan drawn in week 1) and reports the resulting goal probability. The probes name the binding constraint (the cap, the eligible sources, the pack limit, draw timing, or repayment inside the horizon) and classify the case as *reachable*, *limited by policy or sources*, or *not reachable with financing*. They are diagnostics and are never offered as plans; the policy limits do not move.

For the demonstration case the certificate shows that about USD 212,833 of cash in week 1 would suffice within the USD 354,755 cap, yet the best allowed plan reaches 18% and the same cash as free money would reach only 28%: factored invoices repay the factor and the loan matures inside the 13 weeks, so part of the cash leaves again before the low point.

5.11 Learning from outcomes under human control

Memory and attribution. The Core stores each plan with its predicted goal probability and, later, the realised outcome, in a hash-chained case memory. An attribution step classifies a miss as collections timing, order slippage, cost overrun, funding too small, objective too narrow, or unexplained. Retrieval of similar past cases uses standardised Euclidean distance on a small feature vector, with each coordinate divided by its standard deviation across stored episodes.

Estimating a correction. Observed on-time outcomes y_i for buyer b are modelled with the marginal on-time probability $\mu_i(\delta) = \sigma(\text{logit}\, p_i + \delta) \cdot [(1 - \pi) + \pi m]$ (the slowdown state is marginalised out). The learner finds the maximum a posteriori shift under a zero-mean Gaussian prior of precision $\gamma_s = 1$,

$$\delta_b^* = \operatorname{argmax}_\delta \{ \textstyle\sum_{i \text{ in } b} [\, y_i \ln \mu_i(\delta) + (1 - y_i) \ln(1 - \mu_i(\delta)) \,] - (\gamma_s / 2)\, \delta^2 \} \tag{24}$$

by Newton iteration with analytic first and second derivatives, requiring at least 8 observations per buyer and clipping to ±3 logits. The order-shipment scale is a conjugate-style posterior mean with pseudo-count $n_0 = 10$ at the prior scale s_0: $s = (\text{shipped} + n_0 s_0) / (\sum c_i + n_0)$.

Acceptance test. A candidate is evaluated by leave-one-episode-out cross-validation [25]: fit on all other episodes and score the held-out episode by mean log-likelihood per observation, for both the candidate and the active parameters. With K episodes it is accepted only if the mean gain is at least 0.002 nats per observation, it wins on at least ceil(K/2) held-out episodes, and there are at least three episodes and 60

observations. Acceptance only produces a *pending* proposal; activating it is an L2 action that needs a named human, and every activation records who approved it and the evidence.

5.12 Executive Inbox routing

After each event (for example a cancelled order) the Core re-plans and decides whether the Director must be interrupted. An event is routed to the Director only if a *reason* exists: the plan in force has become unavailable; the recommended plan changes *and* the plan in force has lost more than 5 points of goal probability; the goal has become unreachable and the plan in force has fallen; or the first negative week has moved earlier. Otherwise it goes to a daily digest. Within the inbox, events are ordered by

$$score = (\Delta trough / 1000) \cdot certainty / max(1, weeks\ to\ first\ negative\ week) + 10 \cdot I(plan\ in\ force\ loses \geq 30\ points) \quad (25)$$

where Δtrough is the base-case drop in the minimum closing balance and *certainty* is a source-specific constant. Priority is HIGH when four or fewer weeks remain or the plan loses 30 or more points. The rule is a transparent heuristic, not a fitted model; its quality is measured against a threshold baseline in Section 11.

5.13 Financial-distress screening hint

A logistic model on three ledger ratios is shipped as a coefficient file. For a company with DSO, DIO and gross margin $x_{1..3}$ the score is computed as follows: winsorise each ratio at the training 1st and 99th percentiles; transform it with the signed logarithm $\tau(x) = \text{sign}(x)\ \ln(1 + |x|)$; standardise with the training mean and standard deviation; and apply

$$z = \beta_0 + \sum_{j=1}^{3} \beta_j \cdot (\tau(clip(x_j)) - \mu_j) / \sigma_j, \quad p = \sigma(z) \quad (26)$$

The contribution of each ratio to the log-odds is returned, as is a flag for any input outside the training range. The model follows the classical use of accounting ratios for failure prediction [26, 27] and is trained offline on the Polish Companies Bankruptcy data [28] for a one-year horizon. Its ROC-AUC is about 0.74 at one year and 0.59–0.64 at two to five years, against 0.93–0.96 for a tree model on all 64 ratios; the output is therefore labelled a screening hint, is marked out of domain for the synthetic exporter, and states that it is not a credit decision.

6 Document and data integrity

6.1 Matching and reconciliation

The matching engine is pure Python with no I/O and compares money in integer minor units. Bank reconciliation proceeds in three tiers. **Exact**: the reference names one open invoice and the amount is identical. **Tolerance**: the reference names one open invoice and the gap is a bank fee (at most USD 50) or an FX difference. **Subset-sum**: some combination of at most six of the buyer's open invoices sums to the amount received within the fee tolerance; all solutions are returned, and more than one solution yields NEEDS_HUMAN rather than a guess. A three-way match compares the purchase order, commercial invoice and bill of lading. It returns *BLOCKED* when a document is missing or the parties, seller tax id or currency disagree; *FLAGGED*, with the differences listed, when amount, quantity, package count or ports differ; and *ELIGIBLE* only when everything agrees.

Subset-sum step. Finding a set of open invoices whose total equals a received amount, within a tolerance of USD 50, is an instance of the subset-sum problem, which is NP-complete in general [29]. Bounding the subset size at six makes it polynomial: with m open invoices for a buyer the search examines at most $\sum_{k=1}^{6} C(m, k)$ combinations (60,459 for $m = 20$), each tested by $| \sum_{j \text{ in } S} V_j - \text{received} | \leq \text{tolerance}$. All solutions are returned. A unique solution is booked automatically, whereas several equally valid solutions mean the data cannot decide and the match is escalated to a person; the design prefers an explicit abstention to an

arbitrary tie-break.

6.2 Data reliability gate

Four actions are classed as financing actions: preparing a pack, approving a release, running the Autopilot in execute mode, and a lender recording an approval. Each calls the gate first. When the gate is on, an action proceeds only if a server-held quality manifest assesses as *READY_FOR_DRAFT*: all required sources (bank statement, receivables ageing, ledger) present, fresh, reconciled, and independently verified. Verification never comes from the manifest's own "verified by" text. It comes from a signed attestation, an HMAC-SHA256 over the source id, type, as-of date and, when available, the content hash, produced with a server-held key and subject to expiry. Altering the as-of date or content after signing invalidates the attestation. In PRODUCTION the gate is always on and any read or parse failure blocks the action. A pass is not a credit decision and does not prove the underlying document is authentic; the attestation proves only that the key holder vouched for it.

Concretely, an attestation record carries a tag computed over the canonical JSON of its fields r (source id, type, as-of date, content hash, expiry) with the server-held key K_{att}:

$$\sigma(r) = \text{HMAC-SHA-256}_{K_{att}}(\text{canon}(r)) \tag{27}$$

Verification recomputes the tag and compares it in constant time. HMAC is a pseudorandom function under standard assumptions on the compression function [30, 31], so an adversary without K_{att} cannot produce a valid tag for an altered as-of date or content hash. The scheme gives integrity and origin authentication relative to a shared key; it does not give non-repudiation, which would need an asymmetric signature.

6.3 Duplicate-financing control

Each financed invoice becomes one block in a hash-chained registry whose key is canonical, so re-saving a PDF cannot create a second key for the same invoice. A period lock seals a reconciled period with SHA-256; a later change creates version n+1 chained to the previous version rather than editing in place. Both mechanisms are tamper-evident but not tamper-proof, and a registry kept inside one SME installation cannot detect the same invoice being financed at another lender. A shared registry (for example run by a lender consortium) would be required in production.

6.4 Click-to-source provenance

For each figure in a locked pack the provenance module returns: the formula and the same formula with actual inputs substituted; the value recomputed now and whether it reproduces the locked value; the sources as input file, JSON path and current file SHA-256; and, for values read from a scanned document, the document id, page, line, character span and review status. If an input changes after locking, the pack keeps its value and the lineage reports the divergence; it never silently re-locks. Because scans are text layers, fields are located by character offsets and no bounding box is shown. Corrections follow a separate path: a preparer proposes a change with a reason, a different person approves or rejects, the original value and decision are retained, and the correction applies only to the next pack.

6.5 Input defences

Control	Mechanism
Prompt-injection guard	External text is wrapped in fixed delimiters with a standing note; instruction-like patterns are scanned and flagged SUSPICIOUS_CONTENT. The guard never decides an action; the state machine does.
Number lock	Every number in a drafted text must equal a locked metric (after format normalisation). A draft with any other number is rejected and a template is used.
BEC detector	Blocks an email that requests a change of bank details, comes from a look-alike domain (Levenshtein distance at most 2, homoglyph or alternative TLD), or quotes an unverified account. It reads the contact directory and never updates payee data.
PII masking	Replaces account numbers, SWIFT codes, tax ids, emails, phone numbers and known contact names with reversible tokens before any text could reach a cloud model; the token map stays in process memory and is not logged.

Table 7. Defences against untrusted content.

Look-alike domain test. The BEC detector measures the edit distance between a sender domain and each verified contact domain with the Levenshtein recurrence [32]:

$$d(i,j) = \min\{\, d(i-1,j) + 1,\ d(i,j-1) + 1,\ d(i-1,j-1) + I(s_i \neq t_j) \,\} \tag{28}$$

It is computed by dynamic programming in $O(|s||t|)$ time, with $d(i, 0) = i$ and $d(0, j) = j$. A domain that differs from every verified domain but lies within distance 2 of one is blocked, as is a homoglyph substitution or the same name under another top-level domain. A wholly unknown domain is only marked suspicious, which keeps the false-block rate low for legitimate new contacts.

7 Contract and document intelligence

The contract module turns unstructured commercial text into structured, reviewable evidence and produces unsigned drafts. It is a decision-support layer for the SME and its lender, not a legal service: every output carries the status *REQUIRES_HUMAN_REVIEW* or *UNSIGNED_DRAFT_REQUIRES_LEGAL_REVIEW*, and none of the four capabilities below sends data to a language model. The capabilities are (i) bounded text extraction with local OCR, (ii) rule-based risk analysis of contract text, (iii) reconciliation of a contract against an invoice, and (iv) deterministic drafting with Word export. A fifth component, the labelled-field document agent of Section 7.3, serves the invoice, purchase-order and bill-of-lading checks that feed the financing gate.

7.1 Design stance for untrusted documents

A contract, scan or email is an *untrusted input* in the security sense: its content is attacker-influenced, and a financing workflow must not change behaviour because a document says so. Three rules follow throughout the module. First, text is only ever *parsed* by explicit patterns; it is never interpreted as an instruction, and it is never forwarded to a hosted model. Second, every extracted value is a candidate that must be confirmed by a person, so the module reports what it found and what it could not find, together with the evidence line. Third, resource use is bounded before any parsing starts, so a hostile or malformed file fails cheaply.

7.2 Bounded extraction and local OCR

Uploads pass through `contracts/extraction.py`, which handles text, PDF and image inputs in memory only. The limits are fixed in code and are summarised in Table 8.

Control	Value	Purpose
Accepted types	TXT, PDF, PNG, JPG, JPEG	Extension allowlist; other types refused.

Control	Value	Purpose
File size	1 byte to 5 MB	Bounds memory; empty files refused.
PDF pages	At most 10; password-protected refused	Bounds OCR work; no credential handling.
Image size	At most 14 megapixels	Guards against decompression-bomb images.
OCR time limit	12 s per page or image	Prevents a crafted scan from holding a worker.
Output length	Truncated to 50,000 characters, flagged if truncated	Bounds all downstream regular-expression work.
Text-layer threshold	OCR only if a page has fewer than 24 non-blank characters	Prefers exact embedded text over OCR.

Table 8. Extraction limits (contracts/extraction.py).

For a PDF the decision is made *per page*: the embedded text layer is read first, and a page is rendered at 1.5× and passed to the local Tesseract engine [33] only if its text layer holds fewer than 24 non-blank characters. Mixed documents (a typed contract with a scanned appendix) are therefore read with the most reliable method available for each page. The result is labelled with its method (*text*, *pdf-text*, *pdf-ocr-local* or *image-ocr-local*) and always carries the notice that OCR may be inaccurate. If the OCR executable is absent the request is refused with an explanatory message and nothing is guessed; the lean serverless build declines PDF and image parsing in the same explicit way. Because OCR errors in digits are the dominant risk for a financial reader, the later stages treat extracted amounts as evidence to be verified and not as facts.

7.3 Confidence-scored field extraction

The document agent reads labelled documents into typed models. A page marker (`--- page N ---`) tracks the page, and each line of the form *label : value* is indexed with its line number and character span. For every field in a document-type specification (14 fields for an invoice, 11 for a purchase order, 9 for a bill of lading) the label is looked up and the value parsed according to the field kind (text, digits, integer, money, date, currency). Each field receives an ordinal confidence from a fixed ladder:

Confidence	Condition
0.97	Label found and value parsed cleanly.
0.60	Value parsed only after repairing look-alike characters in a numeric field (O→0, l or I→1, S→5, B→8); the note records the repair.
0.30	Label found but the value is unreadable (not a number, date or known currency).
0.00	Label not found.

Table 9. Field confidence ladder (agents/document_agent.py).

Let c_f be the confidence of field f and $\theta = 0.85$ the acceptance threshold. The document-level confidence and the human-review flag are

$$c_{doc} = \min_f c_f \quad NEEDS_HUMAN = I(\min_f c_f < \theta) \tag{29}$$

so a single repaired or unreadable field blocks the whole document from being converted into a typed model. The minimum, not the mean, is used on purpose: a seller tax identifier with one wrong digit is not "92% right"; it is a different identifier. Since $0.60 < \theta$, any repaired value necessarily goes to a person, and the repair note shows what the reader saw. The confidence values are *ordinal rule outcomes*, not calibrated probabilities, and the documentation does not present them as such.

Every accepted field retains its provenance: page, line, character start and end, and the source line (truncated to 300 characters). These coordinates are what the click-to-source panel of Section 6.4 displays, so the chain from a locked figure to the exact characters it was read from is unbroken. Only fully confident

documents are turned into Pydantic models (*CommercialInvoice*, *PurchaseOrder*, *BillOfLading*), with money converted to exact decimal amounts, and passed to the matching engine of Section 6.1. The agent always uses the local rules: a hosted model is never given a document, even if an API key is configured. A defence-in-depth check, retained for an optional model reader, requires that any numeric value claimed by a model literally appears (after removing commas and spaces) in the document text; a value that does not is rejected.

7.4 Rule-based contract risk analysis

The analyser works on at most 50,000 characters of text and returns extracted facts and a list of *flags*. Each fact is obtained by an explicit regular expression, which is the intended behaviour: the rule that produced a value can be read, tested and challenged. The extracted facts are USD amounts (introduced by *USD*, *US$* or *$*), percentages, invoice identifiers (*INV-nnnn*), tax identifiers, whether bank-account wording is present, and *payment milestones*.

Milestone extraction. A percentage is treated as a payment milestone only when it occurs on a line that also contains a payment-event word (payment, paid, deposit, advance, milestone, on delivery, upon acceptance). This restriction is deliberate: a contract also contains percentages for tax, penalties and interest, and counting them would manufacture a spurious "milestones do not sum to 100%" finding. For milestone percentages $p_1, \ldots, p_m$ found at line numbers recorded with the evidence text,

$$\textit{flag PAYMENT_PERCENTAGES if and only if } m \geq 1 \text{ and } |\textstyle\sum_k p_k - 100| > 0.01 \tag{30}$$

The rules that produce flags are summarised in Table 10. Severity expresses how much a reviewer should be interrupted, not a probability of wrongdoing.

Flag code	Severity	Trigger
`UNTRUSTED_INSTRUCTIONS`	high	The prompt-injection scanner of Section 6.5 finds instruction-like text in the document.
`BENEFICIARY_CHANGE`	high	Wording that changes, replaces or switches an account, beneficiary or payment details.
`AMOUNT_MISMATCH`	high	A supplied invoice amount is not within USD 0.01 of any amount stated in the contract.
`VAGUE_TERM`	medium	"As soon as possible", "at our discretion" or "when convenient".
`PAYMENT_PERCENTAGES`	medium	Milestone percentages do not sum to 100%.
`MISSING_PAYMENT_CLAUSE`	medium	No payment wording is recognised.
`TERMINATION_NOT_DETECTED`	low	No termination or cancellation wording is recognised.

Table 10. Contract flags (contracts/intelligence.py).

Invoice cross-check. If an invoice amount v is supplied and the contract states a non-empty set of amounts M, the mismatch rule is

$$\textit{AMOUNT_MISMATCH if and only if } |M| \geq 1 \text{ and } \min_{m \text{ in } M} |m - v| \geq 0.01 \tag{31}$$

The rule abstains when the contract states no amount, because an absent amount is not evidence of a different one. Amounts are compared to the cent, which suits invoices that should match exactly and would be too strict for a contract that states a price excluding tax; the flag is therefore a prompt to investigate and is explicitly labelled a signal and not proof.

Error profile. The rule set is tuned for precision on obvious patterns and relies on mandatory review for recall. Its characteristic failure modes are known: clause wording that avoids the keywords is missed; a percentage on a line that happens to mention "payment" for another reason is counted; amounts written in words, in another currency or without a currency marker are not extracted. The archive contains unit tests

that exercise each flag on constructed inputs (beneficiary change with milestone percentages 35 and 55, an embedded instruction, an amount mismatch) but no precision/recall measurement on a corpus of real contracts, and none is claimed.

7.5 Contract–invoice reconciliation

The reconciliation endpoint analyses the contract, extracts amounts and invoice identifiers from the invoice text, and compares the two sets. With contract amounts A_c and invoice amounts A_i, a discrepancy is declared when both sets are non-empty and no pair agrees within USD 0.01, an $O(|A_c||A_i|)$ search. The union of the contract flags and any mismatch flag is returned with the status *MANUAL_VERIFICATION_REQUIRED*. This text-level check is complementary to the structured three-way match of Section 6.1: the latter compares typed fields from an invoice, purchase order and bill of lading, whereas this check compares free text against a contract and works even when no structured scan exists. Neither can detect the same invoice being financed at another institution, a limit that is stated in the output.

7.6 Deterministic agreement drafting and Word export

Drafting is a pure function from validated inputs to text; no model completes any clause. This is a deliberate liability decision. A model-written contract can contain a plausible but wrong remedy, governing law or number, and a reviewer cannot tell which sentences were generated from the inputs. A template engine guarantees that every figure in the draft is the result of arithmetic on the inputs and that every clause the system cannot responsibly write is visibly left open.

Inputs and validation. The contract type is one of *supply*, *services* or *construction*. Party names are bounded at 120 characters and the scope at 1,200; the currency is one of USD, VND, CNY or EUR; the amount must be a finite number in $(0, 10^{10}]$; and there must be between 1 and 8 payment milestones, each with a percentage in (0, 100]. The milestone percentages must sum to 100 within 0.001, otherwise the request is rejected. The default schedule is 30% on signing, 40% on delivery and 30% after written acceptance. For contract value A and milestone percentage p_k the displayed milestone amount is

$$a_k = \mathrm{round}(A \cdot p_k / 100, 2), \quad \textstyle\sum_k p_k = 100 \tag{32}$$

Rounding each milestone to the cent means the displayed amounts can differ from A by at most 0.005 per milestone in total; the percentage constraint, not the rounded sum, is what is enforced.

Clause skeleton. The output has eleven numbered clauses. Those fully determined by the inputs are written out (parties, scope, price, milestones); the rest carry explicit placeholders that name who must act, so that no clause appears finished when it is not.

Clause	Content	State in the draft
1 Parties, 2 Scope	Names and scope supplied by the user.	Filled from inputs.
3 Price	Currency and total amount; tax treatment left open.	Filled; tax to be confirmed.
4 Payment milestones	Event, percentage and computed amount per milestone.	Filled from validated milestones.
5 Delivery and acceptance	Dates, specifications, evidence of acceptance, remedies.	[REQUIRES NEGOTIATION]
6 Invoicing and late payment	Deadlines, due dates, late fees, disputes.	[REQUIRES NEGOTIATION]
7 Confidentiality and data protection	Appropriate terms.	[APPROPRIATE TERMS REQUIRED]
8 Warranties and liability	Scope, limits, exclusions.	[LEGAL REVIEW REQUIRED]

Clause	Content	State in the draft
9 Termination and disputes	Grounds, notice, remedies, forum.	[LEGAL REVIEW REQUIRED]
10 Governing law	Supplied, otherwise a stated default.	"To be confirmed by counsel" unless provided.
11 Signatures	Signature blocks.	[NOT SIGNED]

Table 11. Clause skeleton of the generated draft (contracts/intelligence.py).

The document begins with *DRAFT — FOR HUMAN LEGAL REVIEW ONLY* and is returned with a five-item review checklist: confirm party identities and signatory authority, confirm applicable law and mandatory clauses, negotiate acceptance, cancellation and dispute rules, verify bank details independently before paying, and obtain approval from qualified counsel before signature.

Word export. The export endpoint calls the same drafting function, so the Word file cannot contain anything the validated draft does not. The document is built with python-docx: a title and subtitle stating *unsigned contract draft, human legal review required*, one second-level heading per numbered clause, the review checklist as a bulleted list, and a closing line that no clause has been reviewed by counsel. There is no electronic-signature path anywhere in the system. Access requires the *sme:analyze* permission, and the endpoint only serialises; it stores nothing.

7.7 Policy-driven contract-to-funding workflow

Contract results can feed the financing analysis through the *decision copilot*, which runs a fixed, read-only chain of local engines selected by an *explicit workflow identifier* and by the *presence of typed input fields*. The conversational question is displayed but is deliberately not an argument to the selection function. Let w be the workflow (one of *auto*, *financial_health*, *contract_check*, *project_finance*, *contract_to_funding*) and let φ be a vector of five booleans recording which of contract text, invoice text, budget, project and cash-waterfall inputs are present. The tool set is

$$tools = ORDER \cap S(w, \varphi), \quad ORDER = (\text{contract_review, invoice_reconcile, financial_eda, budget_scenario, bid_risk, cash_waterfall, funding_compare}) \tag{33}$$

The seven tools run in the fixed order shown, and the set S is given in Table 12. An unknown workflow, a wrongly typed field or a missing prerequisite (for example *contract_to_funding* without both a contract and a project or schedule) is refused with an error before any tool runs.

Workflow	Prerequisites	Tools selected
`auto`	None	financial_eda always; contract_review if contract; invoice_reconcile if contract and invoice; budget_scenario if budget; bid_risk if project; cash_waterfall if schedule; funding_compare if project or schedule.
`financial_health`	None	financial_eda; budget_scenario if budget.
`contract_check`	Contract text	contract_review; invoice_reconcile if invoice text.
`project_finance`	Project or schedule	financial_eda, funding_compare; plus bid_risk, cash_waterfall, budget_scenario where supplied.
`contract_to_funding`	Contract and (project or schedule)	contract_review, financial_eda, funding_compare; plus invoice_reconcile, bid_risk, cash_waterfall, budget_scenario where supplied.

Table 12. Workflow selection (agents/decision_copilot.py).

Why free text cannot widen the tool set. The range of the selection function is a subset of a fixed seven-element set, and its arguments are an enumerated workflow and five booleans. At most $5 \times 2^5 = 160$

argument combinations exist, so there are at most 160 reachable tool sets, all enumerable and testable. Because the question string and the document text are not arguments, text in a contract that says “also transfer funds” cannot change which tools run. This is a non-interference property: the output depends only on the trusted arguments. The summary statements assembled afterwards quote the engines’ results and add the standing caveats (for example, that the APR figure comes from a separate illustrative catalogue and is not project-specific eligibility).

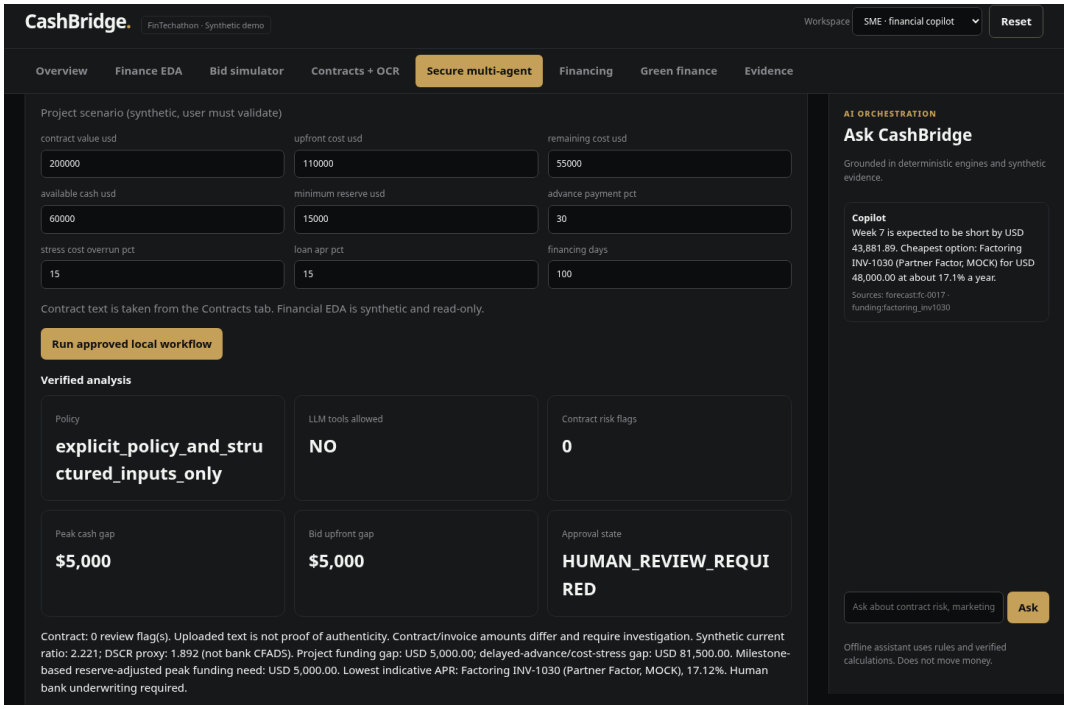


Figure 5. The secure multi-agent workflow screen, from docs/screenshots/10-policy-multi-agent.png (cropped). The upper card states that the question is display-only; the lower panel shows the verified analysis for a synthetic project: a peak cash gap of USD 5,000, "LLM tools allowed: NO", and approval state HUMAN_REVIEW_REQUIRED.

7.8 Scope and verification of the module

- **What it establishes:** amounts, percentages and identifiers present in the text; internal consistency of milestone percentages; agreement between contract and invoice amounts; confidence-scored fields with exact source coordinates.
- **What it does not establish:** legal validity or enforceability, authenticity of a document, absence of fraud, or whether clauses are fair. A flag is a reason to look, not a finding.
- **Data handling:** processing is in memory and local; uploads are not persisted by the analysis endpoints, and no document is sent to a hosted model.
- **Dependencies:** OCR needs a local Tesseract installation (included in the Docker image); PDF reading needs PyMuPDF. When either is absent the system refuses clearly rather than degrading silently.

8 Planning and advisory tools

Besides the 13-week liquidity workflow, the prototype contains a set of deterministic tools that answer the questions an SME asks before it borrows: how healthy is the business, how much can be spent safely, can a project be funded, and what does a green investment do to cash. All of them are read-only, use bounded validated inputs, return *null* instead of an invented value when a denominator is missing, and end with an explicit list of limitations. None is a credit decision.

8.1 Financial ratios and descriptive analysis

The financial summary computes working-capital and solvency indicators from a balance sheet, an annual profit-and-loss account and twelve months of ledger data.

Indicator	Definition	Treatment
DSO, DIO (estimates)	Receivables × 365 / revenue; inventory × 365 / COGS	Year-end balances over annual flow.
DPO, CCC	Unavailable	Left null: trade payables are not separated from other current liabilities.
Current / quick / cash ratio	CA / CL; (CA − inventory) / CL; cash / CL	Null if CL = 0.
Gross, EBITDA margin	(revenue − COGS) / revenue; EBITDA / revenue	EBITDA = revenue − COGS − opex.
Interest cover	EBIT / interest	EBIT = EBITDA − depreciation.
DSCR proxy	EBITDA / (interest + scheduled principal)	Not covenant DSCR or CFADS.
Debt to equity	Interest-bearing debt / book equity	Book values.
Gross cash runway	Cash / average monthly expenses	Assumes zero receipts.

Table 13. Indicators in core/financial_intelligence.py. A ratio is null when its denominator is not positive.

Two choices are worth noting. The cash conversion cycle is *not* computed in this module because DPO cannot be derived honestly without verified trade payables; the 82-day figure in the forecast engine comes from the ledger snapshot that does contain them. And monthly expenses are screened with a population z-score,

$$z_m = (e_m - \mu) / \sigma, \quad \textit{flag month } m \textit{ if } | z_m | \geq 2 \tag{34}$$

with μ and σ the mean and population standard deviation over the twelve months. Under a normal model a two-sided exceedance of 2 occurs with probability $2(1 - \Phi(2)) \approx 0.046$, so about 0.55 of twelve months would be flagged by chance alone; the flag is therefore described as a review prompt and not as fraud detection.

8.2 Budget and marketing-spend scenario

The scenario tests whether a spending decision keeps cash above a protected reserve over a horizon of H months (1 to 12). With historical monthly means of receipts R_{mo}, expenses E_{mo} and marketing K_{mo}, a marketing increase μ percent, a receipts delay δ percent, and a new loan L at rate α percent per annum, the closing balance follows

$$B_k = B_{k-1} + R_{mo} (1 - \delta/100) - [E_{mo} + K_{mo} \mu/100 + L \alpha / 1200], \quad B_0 = cash + L \tag{35}$$

A reserve breach is declared if the minimum balance, including the opening balance, falls below the reserve. The baseline is B_0(without loan) + $H(R_{\text{mo}} - E_{\text{mo}})$. Delayed receipts are excluded from the horizon (a conservative simplification) and not shifted to later months, the loan is interest-only, and principal, fees and collateral are excluded. The tool explicitly does not model marketing return: it says whether the money is affordable, not whether it will pay back.

8.3 Project and bid funding feasibility

For a contract of value V with upfront cost U, remaining cost R, available cash c, protected reserve q, advance payment percentage α, stress overrun o percent, loan rate r and financing period t days, the tool computes deployable cash, the initial funding gap and a stress gap in which the advance is withheld while costs overrun:

$$D = max(\,0,\, c - q\,), \quad G_0 = max(\,0,\, U - \alpha V/100 - D\,), \quad G_1 = max(\,0,\, U(\,1 + o/100\,) - D\,) \tag{36}$$

Aggregated team capacity is handled without reference to individuals: uncovered hours $h^+ = \max(0, H_{\text{req}} - H_{\text{avail}})$ are priced at an outsourced hourly cost w, giving an estimated cost $U + R + h^+ w$, simple financing interest $G_0 r\, t/365$, and

$$profit = V - (\,U + R + h^+\, w\,) - G_0\, r\, t\,/\,365, \quad margin = profit\,/\,V \tag{37}$$

For the demonstration project of Figure 5 (V = 200,000; U = 110,000; c = 60,000; q = 15,000; α = 30%; o = 15%) these formulas give D = 45,000, G_0 = 110,000 − 60,000 − 45,000 = 5,000 and G_1 = 126,500 − 45,000 = 81,500, which are the values the screen displays. The jump from 5,000 to 81,500 is the practical lesson of the tool: a project that looks nearly self-funding on the contractual schedule needs an order of magnitude more cash if the advance is delayed and costs run over. Workforce data is accepted only in aggregate and the tool never scores or ranks employees.

8.4 Milestone cash waterfall

The waterfall replaces the single-number view with a dated schedule. The user supplies receipts and costs as events with a day offset (receipts as an amount or a percentage of contract value, up to 24 receipts and 40 costs). Receipts must total the contract value within $\max(0.05,\, 10^{-6}\, V)$; money that is missing is rejected and never invented. A payment delay of d days shifts every receipt dated after day 0 (the signing receipt is not delayed), and costs are scaled by (1 + overrun/100). With events aggregated by day $t_1 < \ldots < t_n$,

$$B_k = B_{k-1} + \sum receipts(\,t_k\,) - \sum costs(\,t_k\,), \quad P = max(\,0,\, q - B_0,\, max_k\,(\,q - B_k\,)\,) \tag{38}$$

where P is the peak external funding needed to keep the balance at or above the reserve q. An indicative interest figure assumes the peak is drawn for the entire project: $P\, r\, T/365$ with $T = t_n - \min(0, t_1)$. Because the real draw is at most P for at most T days, this is an upper bound on simple interest and is labelled as such. The result is a cash requirement estimate, not lender eligibility; taxes, retentions, guarantees and existing obligations must be modelled separately.

8.5 Capital allocation planner

The capital planner applies the same reserve discipline to a one-off allocation across working capital W, marketing M and green investment G. With cash c and reserve q (required to satisfy $q \le c$):

$$gap = max(\,0,\, W + M + G - (\,c - q\,)\,), \quad gap_{defer} = max(\,0,\, W + G - (\,c - q\,)\,) \tag{39}$$

$$reduction = gap - gap_{defer} \tag{40}$$

The tool also reports headroom when spending is below deployable cash. Arithmetic uses exact decimals with bounds of zero to one billion. It assumes immediate, cash-funded outlays and no new receipts, interest or return, and it states that a positive gap is not a recommended loan size; the instruction to consult the 13-week forecast is part of the output.

8.6 Green-investment pre-screening and cash impact

The green-finance module supports two activity types (rooftop solar and energy-efficient machinery) and has three layers, each deliberately conservative.

Evidence completeness. Each activity type has a checklist of four documents (for solar: equipment quote, site assessment, electricity baseline, generation estimate). A document is reported as *provided_unverified* or *missing*; it is never reported as verified, and an absent item is never read as a positive. Completeness is 100 × k/n for k provided of n required. The result carries fixed fields *impact_is_verified = false*, *green_taxonomy_verified = false* and *bank_credit_approved = false*, and the status is *NEEDS_HUMAN_REVIEW* or *INCOMPLETE*.

Illustrative impact. The energy affected is the expected generation for solar, or baseline consumption times the savings percentage for efficiency. Following the activity-times-emission-factor convention of the GHG Protocol [34],

$$tCO_2e / year = E_{kWh} \times EF / 1000 \tag{41}$$

with the emission factor EF in kg per kWh supplied by the user as an illustrative input. The figure is withheld (null) if the impact assumptions or the factor are missing. A loan request must be positive and no more than the project cost.

Economics and effect on the 13-week forecast. Net annual saving is $S = E_{\mathrm{kWh}} \cdot \text{price} - \text{opex}$ and simple payback is cost/S when $S > 0$ (otherwise undefined). To see what the investment does to liquidity, the module overlays it on the current forecast. The owner-funded part of the capital cost leaves in the chosen week; a green loan pays the supplier directly. For loan principal L = cost × (financed %), monthly rate i = APR/12 and tenor n months, the level instalment is the standard annuity [35]:

$$M = L\, i / (1 - (1 + i)^{-n}), \quad \text{weekly instalment} = 12M / 52, \quad \text{weekly saving} = S / 52 \tag{42}$$

Savings and instalments start four weeks after the capital cost (a commissioning lag). If the cumulative adjustment is Δ_t, the adjusted closing balance is the base forecast plus Δ_t, computed for three cases: no project, cash-funded and loan-funded. For each the module reports the minimum closing balance, the weeks below the buffer, the first negative week, and the extra funding needed to hold the buffer, defined as $\max(0, \text{buffer} - \min \text{closing}_{\text{project}}) - \max(0, \text{buffer} - \min \text{closing}_{\text{base}})$, so it isolates the effect of the project. Tariff paths, degradation and tax are excluded, and the result is explicitly not a taxonomy certification: eligibility under a lender's green criteria, such as the Green Loan Principles [36], requires a separate review. The curated explanation topic *green_investment* repeats this distinction for the user.

8.7 Conversational assistants under a number lock

The CFO assistant answers questions about cash, funding and scenarios, but it *never calculates*. It can call three fixed tools: *query_metric* (one of 16 allowlisted metric identifiers, such as *cash_today_usd*, *dso_days* or *cheapest_option_apr_pct*), *run_scenario* (the forecast re-run under typed what-if controls) and *explain* (the formula, source and data version behind a number). A rule-based router selects one of eight intents (shortfall, cheapest funding, why shortfall, cash conversion cycle, DSO what-if, tariff what-if, FX what-if, buyer risk), so the assistant works with no key and no network. Questions are treated as data: text inside them cannot trigger an action.

When a model is optionally configured to word an answer, its text is subject to the *number lock*. Let $N(x)$ be the set of numbers appearing in a text x after normalisation (thousands separators, a k suffix, currency prefixes and percent signs; dates, document identifiers and source citations are ignored) and let T be the values returned by the tools. The model text is accepted only if

$$N(\text{answer}) - N(T) = \{\} \quad \text{(every number in the answer is a tool result)} \tag{43}$$

and otherwise it is replaced by the template answer. A fabricated figure therefore cannot reach the reader, whatever the model produces. A separate concept explainer accepts one of six predefined public topics (cash conversion cycle, cash reserve, financing APR, contract milestones, financial-fraud signals, green investment), is disabled by default, limits its output to 550 characters, has a local template for every topic, and records *provider_received_private_data = false* in every response.

8.8 Collections: promises, reminders and the forecast

The collections agent closes the loop between buyer behaviour and the forecast. A buyer reply is parsed with a bounded pattern for statements of the form “will pay on <month> <day>”. A date without a year is resolved to its next occurrence on or after the current date, and an impossible date returns no promise. A parsed promise is stored with the source text, and in the scenario model an invoice with a promised date is given on-time probability 1 and the promised week, which removes its payment uncertainty from the simulation without discarding the evidence for it. Reminders follow a ladder at three days before the due date, on the due date, and seven, fifteen and thirty days after, drafted in English, Japanese or Chinese according to the buyer’s market; any number in a draft must pass the number lock, and the first outgoing message to each buyer needs the Director’s action-bound OTP.

9 Security architecture

9.1 Identity, sessions and tenancy

Sessions are stateless HMAC-SHA256 tokens, *base64url(claims).base64url(signature)*, valid for eight hours, so they verify on any instance sharing the session secret. Sign-out, sign-out-everywhere and a failed-login throttle (8 failures per client address and 20 per account in a 300-second window) are held in a SQLite session-state store shared by worker processes. Permissions are checked on the server: *sme:read, sme:analyze, sme:prepare, sme:approve, bank:read, bank:decide*. Hiding a control in the browser grants nothing. Every record carries a tenant, and a lender can see a case only after the Director has released it.

9.2 Tamper-evident audit log

The audit log is an append-only JSONL file in which each line stores the SHA-256 of the previous line over canonical JSON (sorted keys, no whitespace). Editing, reordering or deleting a middle line is detected by chain verification. The limitation is stated in the code: removing the newest lines, or rewriting the whole file, is detected only if the chain head was anchored elsewhere, which a provided script supports but which is not an external anchoring service. Denied actions are logged with the same status as permitted ones.

Formally, if f_n is the content of line n (sequence number, UTC timestamp, case, agent, tool, tier, decision, reason, and the SHA-256 digests of the input and output payloads), then

$$h_n = SHA\text{-}256(\, canon(\, f_n \,\|\, h_{n-1} \,)\,), \quad h_0 = 0^{64} \tag{44}$$

so line n commits to every earlier line through h_{n-1} [37]. Changing, inserting, removing or reordering any interior line changes its hash and therefore every later hash, which chain verification detects and localises to the first inconsistent line. Collision resistance of SHA-256 [31] makes undetected substitution computationally infeasible. Truncating the tail, or regenerating the whole file, yields a valid chain, which is why the head h_N must be anchored outside the host; a script is provided but no external anchor is operated. Payloads are logged only as digests, so the log proves *that* a particular input was processed without storing it.

9.3 Encrypted evidence vault

Stage	Server-side action	Limit
Upload	Type recognised from magic bytes (PDF, PNG, JPEG, UTF-8 text); file name and browser MIME type are not trusted; 10 MB cap; empty files refused; server computes SHA-256.	No malware scanner or content disarm.
Encrypt	AES-256-GCM with tenant, case and file id as associated data, so a ciphertext moved to another case does not decrypt.	Key from environment or a local owner-only file; not an HSM or KMS.
Retrieve	Decrypt, re-hash, compare with the stored hash before returning any byte; no-store download; access audited.	Case-scoped authorisation.
Review	Director releases the case; the lender must open the current file version before the evidence can be accepted.	Version-specific acceptance.
Withdraw	Before the lender decides, revoke access and delete internal case evidence.	Copies already downloaded cannot be recalled.

Table 14. Evidence custody lifecycle (workspace/evidence_vault.py).

Encryption is authenticated encryption with associated data [38]. For plaintext m, a fresh random 96-bit nonce v and associated data AD = tenant ‖ case ‖ file id:

$$(c, \tau) = \text{AES-256-GCM.Enc}_K(v, m; AD), \quad blob = v \,\|\, c \,\|\, \tau \tag{45}$$

Decryption fails unless the authentication tag τ verifies over (v, c, AD). Binding the identifiers into AD means a ciphertext copied to another tenant, case or file fails authentication even though the key is the same. After decryption the SHA-256 of the plaintext is recomputed and compared with the digest stored at upload, which also detects corruption in the store. Random 96-bit nonces under one key remain safe for roughly 2^{32} encryptions before the birthday bound on nonce collision becomes material [38], far above the volume of a case vault.

9.4 Lender integration boundary

A lender adapter layer exposes one neutral quote format behind any number of providers. The default adapter is a mock whose quotes are labelled *simulated*; a sandbox HTTP adapter exists and is exercised only against a mock transport. Every adapter must obey the same contract: a quote carries an expiry and cannot be accepted after it; requests carry an idempotency key derived from the pack's bundle hash, with one retry under the same key; malformed or mismatched provider responses are refused rather than repaired; webhooks must carry a valid HMAC-SHA256 over *timestamp.body*, be at most five minutes old, and are processed once with status moving only forward. A simulated quote is never presented as a lender offer.

9.5 Privacy boundary and data lifecycle

Uploaded contracts, balances, bank details and invoice content stay within the local FastAPI workflow. An optional hosted language service accepts only a predefined public-topic identifier; it receives no free text from users, no case data, and has no tool access, and it is disabled by default. The data lifecycle module inventories each store, exports data on request, and purges by retention policy: audit lines are kept as a legal record with payloads hashed; notifications are deleted after 90 days by default; automation payloads are redacted (hash kept) after retention for non-accounting kinds; and a Director's withdrawal of a release removes internal case evidence. Statutory retention periods differ by jurisdiction and must be confirmed with counsel before production.

The policy boundary between untrusted user text, local analysis agents, human approval and the optional public-topic explainer is shown in Figure 6.

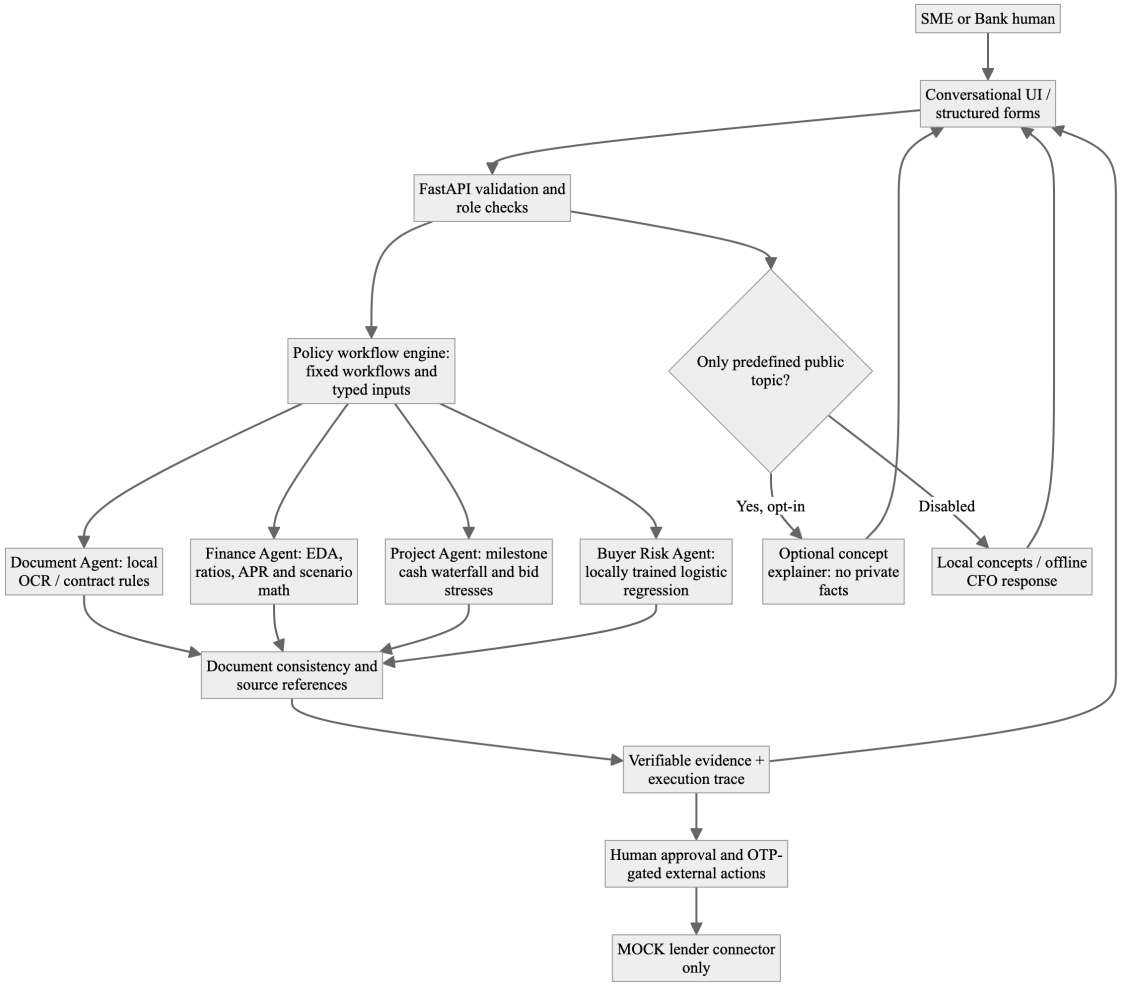


Figure 6. Privacy-first agent architecture, reproduced from docs/PRIVACY_FIRST_AGENT_ARCHITECTURE.md (Mermaid source rendered unchanged apart from label line breaks). User free text selects no tool and no workflow; the language service accepts only a predefined public topic and is disabled by default.

10 Interfaces and workflows

10.1 REST interface

The API groups about 90 routes by purpose. Representative groups are listed below; all routes other than the health, configuration and public overview endpoints require a signed session.

Group	Representative routes	Notes
Authentication	`/api/auth/login, /me, /logout, /logout-all`	Throttled; demo login only in DEMO mode.
Case and forecast	`/api/snapshot, /api/simulate, /api/analytics/summary`	Read-only analysis of the active case.
Agentic	`/api/autopilot/run, /api/intelligence/plan, /simulate-outcome, /propose-update, /decide-update`	Planner output passes validate_plan; updates need a human.
Pack lifecycle	`/api/choose_option, /prepare_pack, /request_otp, /approve, /reject_pack, /withdraw_release`	Preparer and approver roles are distinct.
Bank portal	`/api/bank/inbox, /bank/cases/{id}, /verify, /decision`	Released cases only; hash verification before decision.

Group	Representative routes	Notes
Workspace	`/api/workspace/cases/{id}/evidence, /messages, /requests`	Versioned, encrypted, audited.
Data quality	`/api/workspace/data-gate, /workspace/quality`	Gate status and quality manifest assessment.
Contracts and projects	`/api/contracts/analyze, /draft, /reconcile, /api/projects/waterfall`	Local rules; drafts are unsigned.

Table 15. API route groups (api/main.py).

10.2 End-to-end case

A complete case crosses three people and several security boundaries. (1) The accountant opens the case and sees the first negative week. (2) Sources and invoices are verified by three-way match and the data gate. (3) Options are priced and stress-tested. (4) The accountant locks a pack with versioned sources and evidence identifiers. (5) The Director requests approval and supplies an OTP bound to the exact payload. (6) The coordinator rechecks state and scope and releases only to the authorised lender. (7) The lender opens the current encrypted evidence version. (8) The lender records a mock decision visible to the SME. Fail-closed branches include a data mismatch that stops the pack, an unauthorised account or a wrong or replayed OTP that releases nothing and appends a denial to the audit chain, and a consent withdrawal that removes the reviewer's access.

10.3 Client

The React client is organised by feature (cash forecast, funding, decision centre, autopilot, contracts, green finance, collaboration, bank case review, settings) over shared UI, chart and hook layers. It supports English, Vietnamese and Chinese interfaces and light and dark themes. The client only renders server results; it holds a session token in session storage and display preferences in local storage, and clears the former on sign-out.

10.4 Delivered screens

The figures below are the screenshots shipped with the project README, taken from the running client against synthetic data. The interface is available in English, Vietnamese and Chinese with light and dark themes, and these captures use the Vietnamese dark theme; labels are translated in the captions. Each screen exposes a mechanism described earlier, so the figures serve as evidence of what the system shows a user, not as decoration.

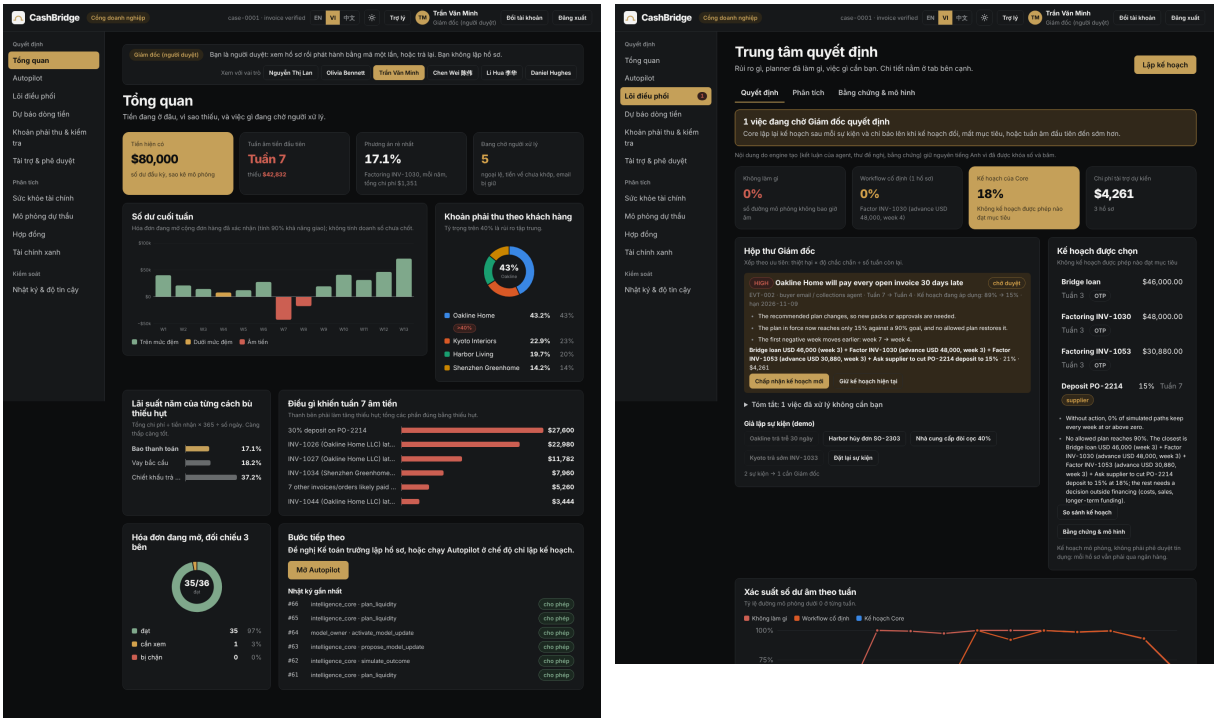
Figure 7(a) is the SME overview. Its tiles state the opening cash (USD 80,000), the first negative week (week 7, short by about USD 42,832), the cheapest bridging option (factoring of INV-1030 at 17.1% a year, total cost USD 1,351) and five items waiting for a decision. The weekly balance chart colours weeks by status (above buffer, below buffer, negative), the donut flags a buyer holding 43.2% of receivables against the 40% concentration threshold, and a bar list attributes the week-7 shortfall to its drivers, led by the 30% supplier deposit on PO-2214 (USD 27,600). The three-way match panel reports 35 of 36 open invoices passing, and the audit feed lists recent tool calls with the gatekeeper decision beside each. Figure 7(b) is the Director's decision centre, where the decision is separated from the analysis: an inbox shows what needs a decision and why, and the recommended plan is listed with its draw week and the approval it needs.

Figure 8(a) is the Intelligence Core of Sections 5.7 to 5.11. It compares three policies on the same simulated paths: no action (8% of paths never run short of cash), the fixed single-advance workflow (29%) and the Core plan (89%), and it states plainly that the 90% goal is not reachable with the allowed plans. The chart of the chance of a negative balance by week shows where each policy fails, and the panel at the foot lists the limits the planner cannot change. Figure 8(b) is the decision trace: five steps (risk detected, documents

checked, scenarios tested, recommendation, approval) each of which opens to its evidence, followed by the feasibility table that relaxes one limit at a time.

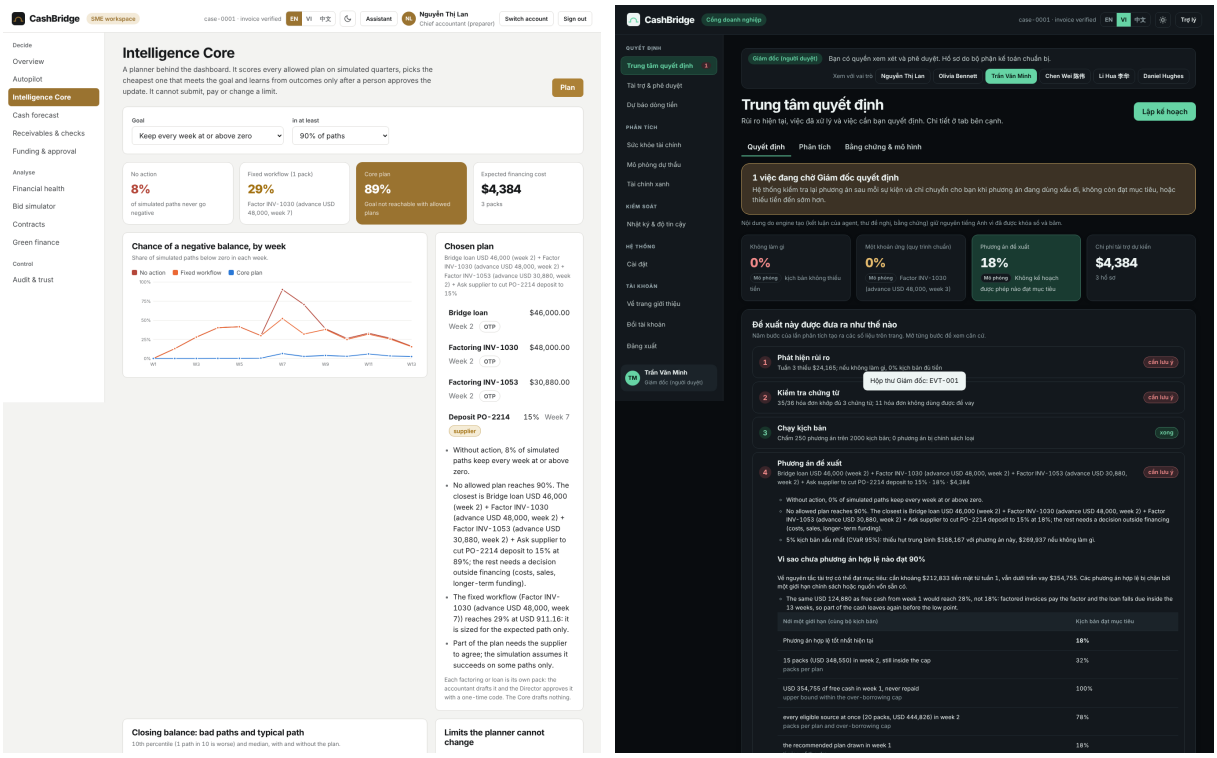
Figure 9(a) is the settings page, which shows to the user the controls the server enforces: the display language and theme, the signed-in account and role, the session and data store, and the approval policy in words (a one-time code valid for 120 seconds, segregation of preparer and approver, the over-borrowing cap, and when the Director is interrupted). Figure 9(b) is a captured end-to-end test: the Director enters a wrong code and the release is refused with the message that the code is not valid for this pack, while the dialog shows the action hash to which the code is bound (Section 4.3).

Figure 10 shows the funding comparison, with the annualised costs of Section 5.5 (factoring 17.12%, bridge loan 18.24%, early-payment discount 37.24%), and the click-to-source panel that opens a locked figure to its formula, recomputed value, source record, scanned-document location and hash (Section 6.4).



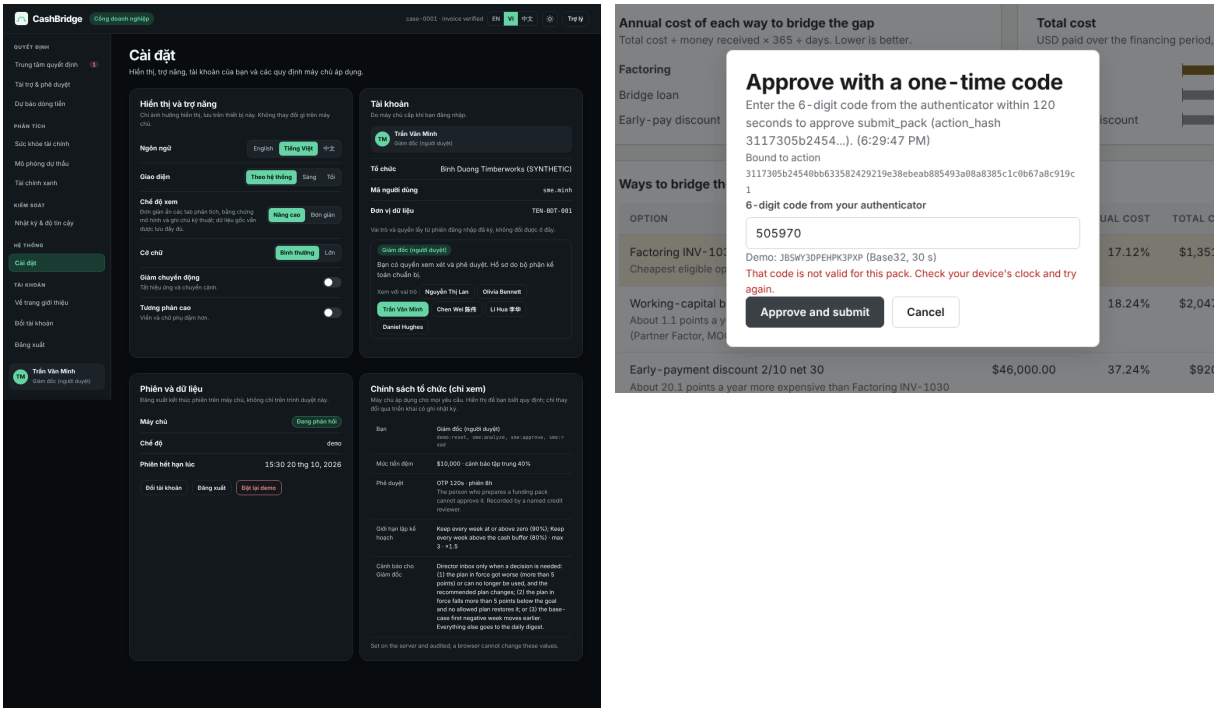
(a) (b)

Figure 7. Delivered client, Vietnamese dark theme, from docs/screenshots/23-sme-overview-vi-dark.png (a) and 32-director-decision-center-vi-dark.png (b, cropped). (a) SME overview: cash, first negative week, cheapest option, shortfall drivers, receivable concentration and three-way match status. (b) Director decision centre: inbox, recommended plan and approvals.



(a) (b)

Figure 8. From docs/screenshots/27-intelligence-core.png (a, cropped) and 40-decision-trace-vi-dark.png (b, cropped). (a) Intelligence Core: goal selector, policy comparison on shared paths, and the chance of a negative balance by week. (b) Decision trace with the feasibility certificate for the unreachable goal.



(a) (b)

Figure 9. From docs/screenshots/42-settings-vi-dark.png (a) and reports/e2e/05_director_wrong_otp.png (b, cropped). (a) Settings and policy page. (b) Release dialog refusing an invalid one-time code; the pack and action hash are shown.

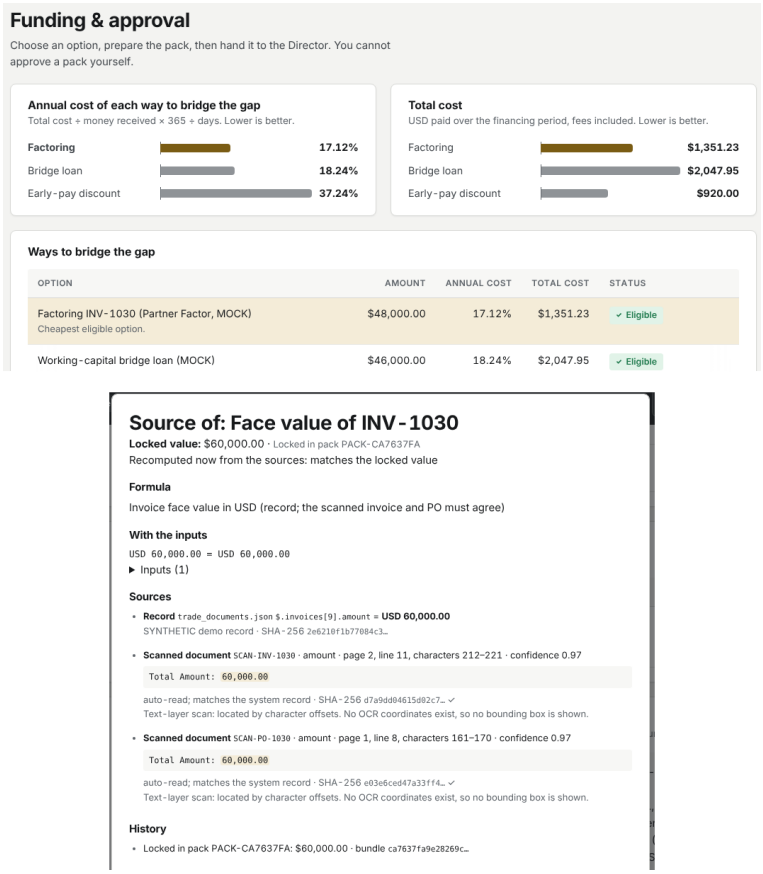


Figure 10. Funding and approval screen with the click-to-source panel for a locked figure, from reports/e2e/17_provenance_click_to_source.png (cropped). The panel shows the formula, the value recomputed from the sources, the contributing records and the scan location.

11 Evaluation

All evaluation is reproducible from the repository (Section 13). Results below are taken from the archived reports and were produced by the team; they are not external certifications. Each is stated with its sample and limitation.

11.1 Metrics and statistical methods

Forecast error is reported as WAPE and bias (Section 5.2) and interval quality as empirical coverage, the fraction of realised values inside the 80% interval, which should be close to 0.80. Mean width is reported with it because an interval can reach any coverage by being wide. Probability models are reported with AUC, Brier score and ECE (Section 5.4). A model is *overconfident* when its mean predicted probability of success exceeds the realised frequency, and the planner benchmark reports that gap in percentage points.

For x events in n trials the Wilson score interval [39] is used in preference to the normal approximation, because it behaves correctly near 0 and 1:

$$p_{L,U} = \left(p + z^2/2n \pm z \sqrt{\left(p(1-p)/n + z^2/4n^2 \right)} \right) / \left(1 + z^2/n \right), \quad p = x/n \tag{46}$$

For $x = 0$ the upper limit reduces to $z^2 / (n + z^2)$. With $n = 145$ and $z = 1.96$ this is 2.6%, the bound quoted for the Executive Inbox: zero misses in 145 needed alerts is consistent with a true miss rate of up to about 2.6%, not with zero. Where observations are clustered (invoices within customers), a customer-clustered bootstrap [40] with 500 resamples is used, resampling whole customers so that within-customer correlation is preserved. Paired differences between plans are computed on the same cases and paths, so the reported intervals are intervals for the difference.

11.2 Functional, security and deployment evidence

Evidence	Result	Limitation
Scenario task suite	20 of 20 pass	Task suite written by the team.
Automated regression	1,609 tests pass (10 in a real browser)	Archived output; regenerate on the submission build.
Adversarial security cases	25 of 25; 5 of 5 agent-governance cases	Cases chosen by the team; not a penetration test.
Production-mode deployment checks	30 of 30 over local HTTPS	Local TLS and worker tests only; no cloud provider.
Demo story	11 of 11 steps hold; 73 hash-chained audit lines; byte-identical across runs	Synthetic case; mock lender.
Queue and outbox fault injection	11 of 11 scenarios hold their invariant	SQLite on one host; mock provider.

Table 16. Functional and security evidence.

11.3 Forecasting and model validation

Metric	Baseline	CashBridge v9	Data and caveat
Receipts WAPE, public invoices	69.9% (v8 rule); 38.1% (contractual)	20.9% (Kaplan–Meier)	2,466 public invoices, 66 rolling origins, 30-day terms only; receipts only.
80% interval coverage (public)	n/a	80.6% split; 81.0% ACI	Same sample.
Receipts WAPE, synthetic history	46.4% (v8); 44.9% (contractual)	42.6%	185 synthetic invoices; ACI coverage 74.0% is below nominal.
Buyer model, unseen customers	AUC 0.50 (v8)	AUC 0.896; ECE 0.040	Temporal split plus unseen customers; public sample, not Vietnamese or Chinese SMEs.
Distress screen, 1-year	Altman Z″ AUC 0.65–0.76 (all horizons)	AUC 0.59–0.74 (ledger ratios)	43,405 Polish company-years; bankruptcy, not loan default.

Table 17. Forecast and model results on external and synthetic data (reports/forecast_backtest.md, external_*_validation.md).

The synthetic-history figures show a smaller gain than the public sample and an ACI coverage under nominal; the documentation reports these rather than selecting the favourable result. The SBA loan-default evaluation was skipped because its data file was not present, and is recorded as not evaluated.

11.4 Planning and decision-routing results

Metric	Reference	CashBridge	Validity
Probability of no negative week (hidden world)	8.7% fixed workflow	66.2% planner with memory; 67.0% with learning	40 simulated quarters, 1,000 paths each.
Policy violations	6 (fixed workflow)	0	Same set.
$CVaR_{95}$ of shortfall (risk benchmark)	USD 99,031 (cheapest plan)	USD 97,461 (risk-aware)	40 synthetic quarters, out-of-sample paths; difference USD 1,571, 95% CI −2,491 to −702; no real-savings claim.
Director alerts, 100 synthetic events	100 (every event); 19 (amount threshold)	7; 0 of 4 needed missed	One seed; labels written by the generator.
Needed alerts missed, 30 seeds	35 of 145 (threshold rule)	0 of 145 (Wilson 95% upper bound 2.6%)	Same generator family; real events need a shadow period.

Table 18. Planner and operations benchmarks (reports/BENCHMARKS.md, risk_planner.md, ops_robustness.md).

Two honest readings accompany these numbers. First, planning for bad paths raises the chance of avoiding a negative week, at roughly four times the expected financing cost of the fixed workflow. Second, learning mainly makes the stated probability truer rather than raising success, because the allowed plans are capped at three packs; and plan stability under small input noise is modest, since near-equal plans can swap.

11.5 What the evidence does not show

- Simulation is not field validation. Forecasts were tested on a public invoice sample and a synthetic history; this does not establish whole-company cash forecasting or loan-default prediction.
- Passing a defined set of adversarial cases does not prove the absence of vulnerabilities.
- User feedback is qualitative. The team reports that participants found the system easy to use and requested minor interface revisions; de-identified records are not part of the submitted archive, so no participant counts, usability scores or completion rates are claimed.

12 Threat model and residual risk

The table lists the principal threats, the implemented mitigation and the remaining exposure. Severity is an engineering prioritisation for this prototype and not a formal quantified rating. Residual risk is not zero.

#	Threat	Implemented mitigation	Remaining exposure
1	Prompt injection, tool hijack	Fixed workflow state machine; untrusted strings treated as data; off-policy proposals refused and logged.	Local parsing errors.
2	Wrong financial answer	Values from local engines; typed inputs; evidence statuses; human release.	Assumptions and forecasts may be wrong.
3	Cross-tenant or role leak	Signed sessions; server-side RBAC; released-case lender scope; denial tests.	No external IdP or enterprise IAM.
4	Malicious document	Byte-level type check; size cap; encrypted vault; human review of complex scans.	No malware scanner.
5	Tampered or duplicate invoice	SHA-256 provenance; three-way match; duplicate registry; fail-closed gate.	No cross-lender registry.
6	OTP replay, unauthorised release	Action-specific, expiring, single-use code; attempt cap; denial log.	Demo OTP is not institutional MFA.

#	Threat	Implemented mitigation	Remaining exposure
7	Partner API replay	Idempotency keys; signed, time-bounded, once-only webhooks.	No certified real lender connection.
8	Confidentiality at rest and in logs	AES-256-GCM evidence; hashed payloads in audit; no-store interfaces; export and purge tools.	Local secret; no KMS or HSM.
9	Model shift and false confidence	Calibrated intervals; probability bounds; published backtests; risk caps; human review.	No real credit-default validation.
10	Revoked data still held	Revoke access; delete internal evidence; record recipient notice.	Earlier downloads need recipient deletion.
11	Infrastructure or log compromise	Fail-closed production mode; hash-chained denial log; TLS and CSP checks.	No independent pentest; no external anchoring.

Table 19. Residual risk register.

13 Reproducibility

Every figure in Section 11 can be regenerated. Seeds, sample sizes and pinned data hashes are recorded in the evidence index (`reports/BENCHMARKS.md`). The public invoice sample is downloaded and hash-verified at run time and is never redistributed.

```
python -m pytest -q                          # unit, integration and adversarial tests
python eval/run_eval.py                      # 20 scenario tasks
python eval/forecast_backtest.py             # rolling-origin receipts benchmark
python eval/external_buyer_validation.py     # buyer model on public invoices
python eval/risk_planner_benchmark.py --cases 40
python eval/ops_robustness.py --seeds 30     # decision-routing benchmark
python scripts/verify_deployment_local.py    # 30 production-mode checks
make evidence                                # regenerate every report
```

Tests cover, among other areas: the gatekeeper and OTP (`test_gatekeeper_otp`, `test_policy_security`); roles and tenancy (`test_auth_rbac`); the vault (`test_evidence_vault`); provenance and the gate (`test_provenance`, `test_financing_gate`); the funding schedule and planner (`test_funding_schedule`, `test_risk_planner`); and real-browser journeys under `tests/e2e`. The submitted archive should be accompanied by regenerated logs from the exact submission build.

14 Limitations and future work

- **Domain.** Models are validated on a public invoice sample and synthetic SME cases. A pilot with real ledgers and a shadow period for the Executive Inbox is required before any claim about real firms.
- **Identity and secrets.** Built-in demo identity, a locally held vault key and TOTP are replaced in production by an external identity provider with institutional MFA, and by a KMS or HSM.
- **Lending.** Lender interaction is mock. A certified connection, KYC/AML process and legal review of the recourse and assignment terms are out of scope.
- **Assurance.** An independent penetration test, malware scanning for uploads, and external anchoring of the audit-chain head would strengthen the integrity claims.
- **Duplicate financing.** A registry shared across lenders, or anchored to an e-invoice authority, is needed to detect cross-lender double financing.
- **Statistics.** Adaptive intervals under-covered on the small synthetic history; larger histories and per-segment calibration are future work.

References

[1] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache and O. Ranen, "HOTP: An HMAC-based one-time password algorithm," IETF RFC 4226, 2005.

[2] D. M'Raihi, S. Machani, M. Pei and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, 2011.

[3] E. L. Kaplan and P. Meier, "Nonparametric estimation from incomplete observations," *J. Amer. Statist. Assoc.*, vol. 53, no. 282, pp. 457–481, 1958.

[4] D. R. Cox, "Regression models and life-tables," *J. R. Statist. Soc. B*, vol. 34, no. 2, pp. 187–220, 1972.

[5] B. Efron and C. Morris, "Data analysis using Stein's estimator and its generalizations," *J. Amer. Statist. Assoc.*, vol. 70, no. 350, pp. 311–319, 1975.

[6] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," *Int. J. Forecasting*, vol. 22, no. 4, pp. 679–688, 2006.

[7] V. Vovk, A. Gammerman and G. Shafer, *Algorithmic Learning in a Random World*. New York: Springer, 2005.

[8] J. Lei, M. G'Sell, A. Rinaldo, R. J. Tibshirani and L. Wasserman, "Distribution-free predictive inference for regression," *J. Amer. Statist. Assoc.*, vol. 113, no. 523, pp. 1094–1111, 2018.

[9] K. Stankeviciute, A. M. Alaa and M. van der Schaar, "Conformal time-series forecasting," in *Adv. Neural Information Processing Systems* 34, 2021.

[10] I. Gibbs and E. Candès, "Adaptive conformal inference under distribution shift," in *Adv. Neural Information Processing Systems* 34, 2021.

[11] D. W. Hosmer, S. Lemeshow and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ: Wiley, 2013.

[12] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982.

[13] G. W. Brier, "Verification of forecasts expressed in terms of probability," *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

[14] M. P. Naeini, G. F. Cooper and M. Hauskrecht, "Obtaining well calibrated probabilities using Bayesian binning into quantiles," in *Proc. AAAI*, 2015.

[15] L. F. Klapper, "The role of factoring for financing small and medium enterprises," World Bank Policy Research Working Paper 3593, 2005.

[16] P. Glasserman, *Monte Carlo Methods in Financial Engineering*. New York: Springer, 2004.

[17] R. T. Rockafellar and S. Uryasev, "Optimization of conditional value-at-risk," *J. Risk*, vol. 2, no. 3, pp. 21–41, 2000.

[18] P. Artzner, F. Delbaen, J.-M. Eber and D. Heath, "Coherent measures of risk," *Mathematical Finance*, vol. 9, no. 3, pp. 203–228, 1999.

[19] A. J. Kleywegt, A. Shapiro and T. Homem-de-Mello, "The sample average approximation method for stochastic discrete optimization," *SIAM J. Optimization*, vol. 12, no. 2, pp. 479–502, 2002.

[20] A. Shapiro, D. Dentcheva and A. Ruszczyński, *Lectures on Stochastic Programming: Modeling and Theory*. Philadelphia: SIAM, 2009.

[21] A. Charnes and W. W. Cooper, "Chance-constrained programming," *Management Science*, vol. 6, no. 1, pp. 73–79, 1959.

[22] E. Delage and Y. Ye, "Distributionally robust optimization under moment uncertainty with application to data-driven problems," *Operations Research*, vol. 58, no. 3, pp. 595–612, 2010.

[23] P. Mohajerin Esfahani and D. Kuhn, "Data-driven distributionally robust optimization using the Wasserstein metric," *Mathematical Programming*, vol. 171, pp. 115–166, 2018.

[24] A. Ben-Tal, L. El Ghaoui and A. Nemirovski, *Robust Optimization*. Princeton, NJ: Princeton University Press, 2009.

[25] M. Stone, "Cross-validatory choice and assessment of statistical predictions," *J. R. Statist. Soc. B*, vol. 36, no. 2, pp. 111–147, 1974.

[26] E. I. Altman, “Financial ratios, discriminant analysis and the prediction of corporate bankruptcy,” *J. Finance*, vol. 23, no. 4, pp. 589–609, 1968.

[27] J. A. Ohlson, “Financial ratios and the probabilistic prediction of bankruptcy,” *J. Accounting Research*, vol. 18, no. 1, pp. 109–131, 1980.

[28] M. Zięba, S. K. Tomczak and J. M. Tomczak, “Ensemble boosted trees with synthetic features generation in application to bankruptcy prediction,” *Expert Systems with Applications*, vol. 58, pp. 93–101, 2016.

[29] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York: W. H. Freeman, 1979.

[30] H. Krawczyk, M. Bellare and R. Canetti, “HMAC: Keyed-hashing for message authentication,” IETF RFC 2104, 1997.

[31] NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.

[32] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals,” *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[33] R. Smith, “An overview of the Tesseract OCR engine,” in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[34] World Resources Institute and World Business Council for Sustainable Development, *The Greenhouse Gas Protocol: A Corporate Accounting and Reporting Standard*, rev. ed., 2004.

[35] R. A. Brealey, S. C. Myers and F. Allen, *Principles of Corporate Finance*, 13th ed. New York: McGraw-Hill, 2020.

[36] Loan Market Association, Asia Pacific Loan Market Association and Loan Syndications and Trading Association, *Green Loan Principles*, Feb. 2023.

[37] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” *J. Cryptology*, vol. 3, no. 2, pp. 99–111, 1991.

[38] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, 2007.

[39] E. B. Wilson, “Probable inference, the law of succession, and statistical inference,” *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[40] B. Efron, “Bootstrap methods: another look at the jackknife,” *Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

Appendix A. Agent tool allowlists

Agent	Allowed tools
coordinator	`read_bank_statements, read_documents, read_inbox, read_payment_history, reconcile, run_forecast, rank_funding_options, scan_email, ledger_write, lock_period,` **`submit_pack`**`,` **`send_email`**
document_agent	`read_documents, three_way_match, reconcile, subset_sum, draft_text`
collections_agent	`read_payment_history, schedule_reminders, draft_text, parse_reply`
buyer_risk_agent	`read_payment_history, score_buyer_risk`
cfo_assistant	`query_metric, run_scenario, explain, draft_text`
intelligence_core	`plan_liquidity, simulate_outcome, propose_model_update`
model_owner (human)	`activate_model_update, record_plan_decision`
bank_reviewer (human)	`read_documents, record_credit_decision`
(none)	`transfer_funds, update_payee, make_payment: recognised as L4 and refused for every caller`

Table 20. Allowlists from config/tool_allowlist.py. Only the coordinator holds L3 tools.

Appendix B. Principal configuration constants

Constant	Value	Meaning
FORECAST_WEEKS	13	Forecast horizon.
MIN_CASH_BUFFER_USD	10,000	Below this a week is flagged at risk.
OTP_VALIDITY_SECONDS	120	Challenge lifetime.
OTP_MAX_FAILED_ATTEMPTS	5	Wrong codes before a challenge is voided.
SESSION_TTL_SECONDS	28,800	Session lifetime (8 h).
CVAR_WEIGHT	0.02	Weight of $CVaR_{95}$ in the plan objective.
over_borrowing_factor	1.5	Total funding cap relative to estimated P95 shortfall.
Maximum financing packs	3	Independent guard in validate_plan.
MAX_BYTES (vault)	10,000,000	Upload size cap.
Prior strength k	8	Beta shrinkage for per-buyer payment hazards.
P_FLOOR, P_CEILING	0.02, 0.98	Bounds on buyer on-time probability.
Conformal alpha, gamma	0.2, 0.02	80% intervals; ACI step size.

Table 21. Constants as defined in config/settings.py and the modules named in the text.

Appendix C. Repository layout

```
agents/       coordinator, autopilot, intelligence_core, specialist agents
api/          FastAPI app, lender adapters, mock lender connector
automation/   job engine, crash-safe queue with retry and dead-letter handling
config/       settings, runtime mode, tool allowlist
core/         forecasting, payment timing, conformal, funding, simulation,
              matching, provenance, gates, registry, lifecycle
security/     auth, gatekeeper, OTP, audit log, input defences
workspace/    store, encrypted evidence vault, corrections
schemas/      typed agent, document and forecast schemas
frontend/     React 18 + TypeScript + Vite client
eval/         benchmark and validation harnesses
reports/      archived benchmark, deployment and end-to-end evidence
tests/        unit, integration, adversarial and browser tests
```